ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO BÀI TẬP LỚN

MÔN: HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

MÃ LỚP HỌC PHẦN: INT3202 3

ĐỀ TÀI B6:

NGHIÊN CỨU KIẾN TRÚC VÀ CƠ CHẾ HOẠT ĐỘNG CỦA HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU PHÂN TÁN

Giảng viên hướng dẫn: GVC. ThS. Vũ Bá Duy

Nhóm sinh viên thực hiện: Nhóm 11

1. Lê Bá Sơn (Nhóm trưởng) MSSV: 24020294
2. Lê Bá Minh Hiếu MSSV: 24020125
3. Nguyễn Văn Tiến MSSV: 24020321

HÀ NỘI – 2026

Lời Mở Đầu

Trong bối cảnh dữ liệu số gia tăng đột biến và các hệ thống dịch vụ trực tuyến đòi hỏi tính sẵn sàng cao cùng khả năng phục vụ toàn cầu, các hệ quản trị cơ sở dữ liệu tập trung truyền thống ngày càng bộc lộ những hạn chế rõ rệt về hiệu năng truy xuất và khả năng mở rộng. Để vượt qua các rào cản đó, **Hệ Quản trị Cơ sở Dữ liệu Phân tán (Distributed Database Management System – DDBMS)** đã ra đời và phát triển thành nền tảng trọng yếu trong thiết kế kiến trúc phần mềm hiện đại.

Báo cáo bài tập lớn này được thực hiện trong khuôn khổ học phần **Hệ Quản trị Cơ sở Dữ liệu** nhằm nghiên cứu chuyên sâu và toàn diện về **Đề tài B6: Nghiên cứu kiến trúc và cơ chế hoạt động của hệ quản trị cơ sở dữ liệu phân tán**. Toàn bộ nội dung tập trung làm rõ 3 trụ cột lý thuyết cốt lõi:

1. Tổng quan kiến trúc hệ thống, kiến trúc phân tầng ANSI/SPARC mở rộng và các kỹ thuật phân mảnh dữ liệu.
2. Quy trình xử lý và chiến lược tối ưu hóa truy vấn phân tán xuyên mạng (đặc biệt là kỹ thuật Semi-join).
3. Quản lý giao dịch, các giao thức cam kết phân tán (2PC, 3PC), kiểm soát đồng thời và bài toán đánh đổi theo định lý CAP.

Nhóm chúng em xin chân thành cảm ơn thầy **GVC. ThS. Vũ Bá Duy** đã tận tình giảng dạy và cung cấp những nguồn tài liệu tham khảo chuẩn mực trong suốt thời gian qua.

Bảng phân công nhiệm vụ của từng thành viên:

STT	Họ và tên	Mã SV	Nội dung phụ trách chính
1	Lê Bá Sơn (NT)	24020294	Tổng quan, Mô hình kiến trúc DDBMS và Kỹ thuật phân mảnh dữ liệu
2	Lê Bá Minh Hiếu	24020125	Quy trình xử lý và Tối ưu hóa truy vấn phân tán (Semi-join)
3	Nguyễn Văn Tiến	24020321	Quản lý giao dịch, Kiểm soát đồng thời và Nhất quán phân tán (2PC, CAP)

Mục lục

Lời Mở Đầu **i**

1 Tổng Quan Về Hệ Quản Trị Cơ Sở Dữ Liệu Phân Tán **1**

1.1 Đặt Vấn Đề và Động Lực Phát Triển 1

1.2 Khái Niệm và Bản Chất của DDBMS 1

1.2.1 Khái niệm Hệ CSDL Phân tán 1

1.2.2 Phân biệt Hệ phân tán và Hệ thống xử lý tập trung 1

1.3 Các Mức Độ Trong Suốt (Transparency) 2

1.3.1 Trong suốt phân mảnh (Fragmentation Transparency) 2

1.3.2 Trong suốt nhân bản (Replication Transparency) 3

1.3.3 Trong suốt vị trí (Location Transparency) 3

1.3.4 Trong suốt giao dịch (Distributed Transaction Transparency) 4

2 Kiến Trúc Hệ Thống Và Kỹ Thuật Phân Mảnh Dữ Liệu **5**

2.1 Kiến Trúc Tham Chiếu Của Hệ Phân Tán 5

2.1.1 Kiến trúc phân tầng ANSI/SPARC mở rộng cho DDBMS 5

2.1.2 Mô hình Client–Server, Collaborating Server và Middleware 6

2.1.3 Hệ phân tán đồng nhất, không đồng nhất và Gateway Protocols . . 7

2.2 Kỹ Thuật Phân Mảnh Dữ Liệu (Data Fragmentation) 8

2.2.1 Phân mảnh ngang (Primary và Derived) 8

2.2.2 Phân mảnh dọc (Vertical Fragmentation) 10

2.2.3 Phân mảnh hỗn hợp (Hybrid Fragmentation) 11

2.3 Các Quy Tắc Bắt Buộc Trong Phân Mảnh 11

2.3.1 Tính đầy đủ (Completeness) 11

2.3.2 Tính tái tạo (Reconstruction) 11

2.3.3 Tính tách biệt (Disjointness) 11

2.4 Chiến Lược Phân Bổ và Cấp Phát Dữ Liệu 12

2.4.1 Bài toán cấp phát dữ liệu 12

2.4.2 Các mô hình cấp phát dữ liệu 12

3 Quy Trình Xử Lý Và Tối Ưu Hóa Truy Vấn Phân Tán 14
3.1 Tổng Quan Về Kiến Trúc Xử Lý Truy Vấn Phân Tán . . . 14
3.2 Chi Tiết Các Bước Xử Lý Câu Lệnh Truy Vấn . . . 14
3.2.1 Phân rã truy vấn (Query Decomposition) . . . 14
3.2.2 Bản địa hóa dữ liệu (Data Localization) . . . 16
3.2.3 Tối ưu hóa toàn cục (Global Optimization) . . . 17
3.3 Bài Toán Định Lượng Minh Họa Phép Toán Semi-Join . . . 18
3.3.1 Bối cảnh và Thông số Kỹ thuật . . . 19
3.3.2 Phân tích So sánh 2 Chiến lược Thực thi . . . 19
3.3.3 Bảng Tổng hợp và Đánh giá Hiệu quả Định lượng . . . 20
3.3.4 Nguyên tắc Áp dụng và Trade-off Kỹ thuật . . . 20
3.4 Phân Tích Tình Huống Xuyên Suốt (Case Study Truy Vấn Đa Trạm) . . . 21
3.4.1 Bối cảnh Hệ thống . . . 21
3.4.2 Diễn tiến qua 3 Bước Xử lý . . . 21
3.5 Tổng Kết Và So Sánh Các Giai Đoạn Xử Lý Truy Vấn . . . 22

4 Quản Lý Giao Dịch Và Kiểm Soát Đồng Thời Phân Tán 24
4.1 Tổng Quan Giao Dịch Và Kiến Trúc Xử Lý Phân Tán . . . 24
4.1.1 Giao dịch phân tán là gì? . . . 24
4.1.2 Các thành phần chức năng . . . 24
4.1.3 Phân biệt giao dịch, cam kết và sao chép dữ liệu . . . 24
4.2 Thuộc Tính ACID Trên Môi Trường Nhiều Máy Chủ . . . 25
4.3 Giao Thức Cam Kết Hai Pha (Two-Phase Commit – 2PC) . . . 26
4.3.1 Mục đích và giả định . . . 26
4.3.2 Pha 1 – PREPARE / VOTING (Chuẩn bị và Bỏ phiếu) . . . 26
4.3.3 Pha 2 – DECISION (Quyết định Cuối cùng) . . . 26
4.3.4 Lưu đồ thuật toán 2PC . . . 27
4.3.5 Bảng trạng thái và thông điệp trong 2PC . . . 27
4.3.6 Ví dụ chi tiết: Giao dịch chuyển tiền giữa hai máy chủ . . . 27
4.4 Xử Lý Lỗi Và Phục Hồi (Recovery) Trong 2PC . . . 28
4.4.1 Vai trò của nhật ký bền vững (Write-Ahead Logging) . . . 28
4.4.2 Phân tích các kịch bản sự cố và khả năng phục hồi . . . 29
4.4.3 Tại sao 2PC là giao thức Blocking? . . . 29
4.4.4 Lưu đồ phục hồi tại Participant . . . 29
4.5 Giao Thức Cam Kết Ba Pha (Three-Phase Commit – 3PC) . . . 29
4.5.1 Động cơ và các giai đoạn . . . 29
4.5.2 Phục hồi và giới hạn kỹ thuật của 3PC . . . 29
4.5.3 Bảng so sánh đối chiếu giữa 2PC và 3PC . . . 32

4.6 Kiểm Soát Đồng Thời Phân Tán 32
4.6.1 Vì sao cần kiểm soát đồng thời? 32
4.6.2 Khóa hai pha phân tán (Distributed Two-Phase Locking – 2PL) . . 32
4.6.3 Điều khiển theo nhãn thời gian (Timestamp Ordering – TO) 33
4.6.4 Phát hiện và xử lý bế tắc toàn cục (Global Deadlock Detection) . . 33
4.7 Định Lý CAP Và Sự Đánh Đổi Trong Thực Tế 33
4.7.1 Phát biểu và cách hiểu chính xác 33
4.7.2 Ví dụ phân hoạch với hai bản sao dữ liệu 34
4.7.3 Bảng so sánh xu hướng thiết kế CP và AP 34
4.7.4 Liên hệ giữa CAP, 2PC và mô hình BASE 35
4.8 Đánh Giá Tổng Hợp Và Đề Xuất Thiết Kế 35

Phụ Lục: Mã Giả Và Bộ Tình Huống Kiểm Thử 36

5 Tổng Kết Và Hướng Phát Triển 39
5.1 Tổng Kết Kết Quả Nghiên Cứu 39
5.2 Những Nhận Định Và Bài Học Cốt Lõi Rút Ra 40
5.3 Định Hướng Triển Khai Thực Nghiệm 40

Chương 1

Tổng Quan Về Hệ Quản Trị Cơ Sở Dữ Liệu Phân Tán

1.1 Đặt Vấn Đề và Động Lực Phát Triển

Trong các ứng dụng quy mô lớn hiện đại, lượng dữ liệu sinh ra gia tăng liên tục cùng nhu cầu phục vụ người dùng trải rộng ở nhiều khu vực địa lý. Các hệ quản trị cơ sở dữ liệu tập trung truyền thống đối mặt với giới hạn mở rộng phần cứng đắt đỏ, rủi ro đơn điểm lỗi, cũng như độ trễ mạng cao khi người dùng ở xa truy cập về máy chủ trung tâm.

Hệ Quản trị Cơ sở Dữ liệu Phân tán ra đời nhằm giải quyết triệt để các hạn chế trên bằng cách mở rộng theo chiều ngang, cho phép kết nối các máy tính phổ thông thành một cụm xử lý thống nhất, tăng cường tính sẵn sàng, khả năng chịu lỗi và tính cục bộ trong truy xuất dữ liệu.

1.2 Khái Niệm và Bản Chất của DDBMS

1.2.1 Khái niệm Hệ CSDL Phân tán

Hệ cơ sở dữ liệu phân tán (Distributed Database System) là một hệ thống gồm tập hợp nhiều cơ sở dữ liệu có mối liên hệ logic chặt chẽ, được phân bố và lưu trữ vật lý trên nhiều nút mạng khác nhau trong một hệ thống mạng máy tính. Toàn bộ các tài nguyên này được quản lý đồng thời bởi các DBMS thành phần có khả năng hoạt động tự trị nhất định.

1.2.2 Phân biệt Hệ phân tán và Hệ thống xử lý tập trung

Sự khác biệt cốt lõi giữa hai hệ thống được tổng hợp trong Bảng 1.1:

Bảng 1.1: So sánh Hệ CSDL Tập trung và Hệ CSDL Phân tán

Tiêu chí	**Hệ CSDL Tập trung**	**Hệ CSDL Phân tán**
Vị trí lưu trữ	Đặt tại một nút mạng trung tâm duy nhất.	Phân mảnh và lưu trữ trên nhiều nút mạng vật lý độc lập.
Bản chất dữ liệu	Nguyên khối tại chỗ, không phân rã logic.	Phân tán vật lý, liên kết logic qua lược đồ toàn cục.
Quản trị	Một DBMS trung tâm điều khiển.	Nhiều DBMS thành phần phối hợp điều phối và đồng bộ.
Chịu lỗi (SPOF)	Điểm nghẽn đơn điểm lỗi; lỗi trạm chính sẽ gián đoạn toàn hệ thống.	Khả năng chịu lỗi cao; một nút bị sập, các nút còn lại vẫn tiếp tục phục vụ.
Khả năng mở rộng	Mở rộng chiều dọc, phụ thuộc giới hạn phần cứng máy chủ.	Mở rộng chiều ngang linh hoạt bằng cách bổ sung nút mới.
Tối ưu truy vấn	Tập trung tối ưu chi phí Disk I/O và tính toán CPU.	Cân đối thêm chi phí truyền thông mạng (băng thông, độ trễ liên trạm).

1.3 Các Mức Độ Trong Suốt (Transparency)

Tính trong suốt là khả năng che giấu các chi tiết kỹ thuật phức tạp liên quan đến hạ tầng mạng vật lý, cấu trúc phân mảnh và cơ chế đồng bộ giữa các trạm, giúp người dùng thao tác như trên một CSDL tập trung duy nhất.

1.3.1 Trong suốt phân mảnh (Fragmentation Transparency)

Là cấp độ trong suốt cao nhất trong tổ chức dữ liệu. Người dùng viết câu lệnh SQL trực tiếp trên các quan hệ toàn cục mà không cần quan tâm bảng gốc bị bẻ nhỏ theo hình thức nào (ngang, dọc hay hỗn hợp). Hệ thống tự động thay thế tên bảng toàn cục bằng các biểu thức tái tạo ($\bigcup$ hoặc $\bowtie$).

Ví dụ: Thao tác SQL trong môi trường trong suốt phân mảnh

Xét CSDL có quan hệ toàn cục NhanVien(MaNV, HoTen, Luong, MaCN). Dữ liệu phân mảnh ngang theo chi nhánh: NV_HN (Trạm 1) và NV_HCM (Trạm 2).
Lập trình viên truy vấn trực tiếp trên bảng toàn cục:

```
SELECT MaNV, HoTen, Luong
FROM NhanVien
WHERE Luong > 15000000;
```

DDBMS ngầm dịch tự động thành truy vấn phân tán thực thi tại các trạm:

```
SELECT MaNV, HoTen, Luong FROM NV_HN WHERE Luong > 15000000
UNION ALL
SELECT MaNV, HoTen, Luong FROM NV_HCM WHERE Luong > 15000000;
```

1.3.2 Trong suốt nhân bản (Replication Transparency)

Tính trong suốt nhân bản đảm bảo người dùng và ứng dụng hoàn toàn không cần biết một quan hệ hay một phân mảnh dữ liệu có tồn tại bản sao hay không, số lượng bản sao là bao nhiêu và chúng đang được lưu trữ tại những nút mạng nào. Dữ liệu nhân bản được trừu tượng hóa như chỉ có một bản duy nhất trên toàn hệ thống.

Cơ chế điều phối ngầm của DDBMS mang lại các ưu thế vượt trội:

- **Tự động tối ưu hóa và phân tải truy vấn đọc:** Khi phát sinh thao tác đọc, DDBMS tự động chọn bản sao có chi phí mạng thấp nhất hoặc nút có tải xử lý nhẹ nhất để phục vụ, giúp gia tăng đáng kể thông lượng truy xuất của toàn hệ thống.
- **Duy trì nhất quán tự động khi ghi dữ liệu:** Khi có thao tác thay đổi dữ liệu, DDBMS tự động điều phối đồng bộ thay đổi tới tất cả các bản sao liên quan thông qua các giao thức kiểm soát nhất quán mà không đòi hỏi tầng ứng dụng phải phát nhiều lệnh ghi riêng biệt.
- **Đảm bảo tính sẵn sàng cao:** Nếu một trạm chứa bản sao gặp sự cố phần cứng hoặc mất kết nối mạng, hệ thống tự động chuyển hướng truy vấn sang các bản sao còn lại, duy trì dịch vụ liên tục.

1.3.3 Trong suốt vị trí (Location Transparency)

Tính trong suốt vị trí đảm bảo người dùng và chương trình ứng dụng có thể truy xuất, thao tác trên dữ liệu mà không cần biết dữ liệu đó đang đặt tại vị trí nào. Mọi ánh xạ từ tên bảng logic sang địa chỉ vật lý thực tế đều do DDBMS tự động phân giải thông qua Danh mục Hệ thống Toàn cục (Global System Catalog).

Giá trị cốt lõi của tính trong suốt vị trí thể hiện qua các khía cạnh:

- **Độc lập dữ liệu vật lý:** Tách rời hoàn toàn tầng logic ứng dụng khỏi hạ tầng mạng. Lập trình viên chỉ cần viết các câu lệnh SQL chuẩn trên lược đồ toàn cục như một CSDL tập trung duy nhất, loại bỏ hoàn toàn việc phải mở nhiều kết nối hay tự viết mã điều hướng luồng truy vấn cho từng máy chủ.
- **Tự do tái cấu trúc và di chuyển dữ liệu:** Quản trị viên có thể linh hoạt di chuyển các phân mảnh giữa các nút mạng hoặc thay đổi địa chỉ IP máy chủ. Khi đó, hệ thống chỉ cần cập nhật System Catalog mà không cần sửa đổi mã nguồn ứng dụng.
- **Tăng cường khả năng chịu lỗi và mở rộng:** Cho phép hệ thống ngầm chuyển hướng truy vấn sang các nút dự phòng hoặc phân tán tải sang các trạm mới thêm vào cụm mà người dùng cuối không hề nhận thấy sự thay đổi về mặt kiến trúc.

1.3.4 Trong suốt giao dịch (Distributed Transaction Transparency)

Tính trong suốt giao dịch đảm bảo rằng một giao dịch phân tán được thực thi một cách trọn vẹn và an toàn mà người dùng hay lập trình viên không cần can thiệp vào các tiến trình điều phối vật lý phức tạp bên dưới.

- **Trong suốt tương tranh:** Cho phép nhiều giao dịch từ nhiều nút mạng thao tác đồng thời trên cùng các tập dữ liệu phân tán nhưng vẫn đảm bảo tính khả tuần tự. Lập trình viên không cần tự viết mã quản lý khóa hay kiểm tra tranh chấp tài nguyên; DDBMS tự động duy trì tính cô lập thông qua các giải thuật và các cơ chế phát hiện bế tắc toàn cục.
- **Trong suốt sự cố và phục hồi:** Đảm bảo tính nguyên tử và tính bền vững. Khi một giao dịch cập nhật đồng thời nhiều trạm, nếu một trạm bị mất điện hoặc đường truyền mạng bị ngắt quãng giữa chừng, DDBMS sẽ tự động kích hoạt cơ chế hủy bỏ và khôi phục đồng bộ trạng thái ban đầu trên toàn bộ các trạm còn lại thông qua các giao thức cam kết phân tán như Cam kết hai pha (2PC) hoặc Ba pha (3PC) mà không để lại dữ liệu rác hay trạng thái bất nhất.

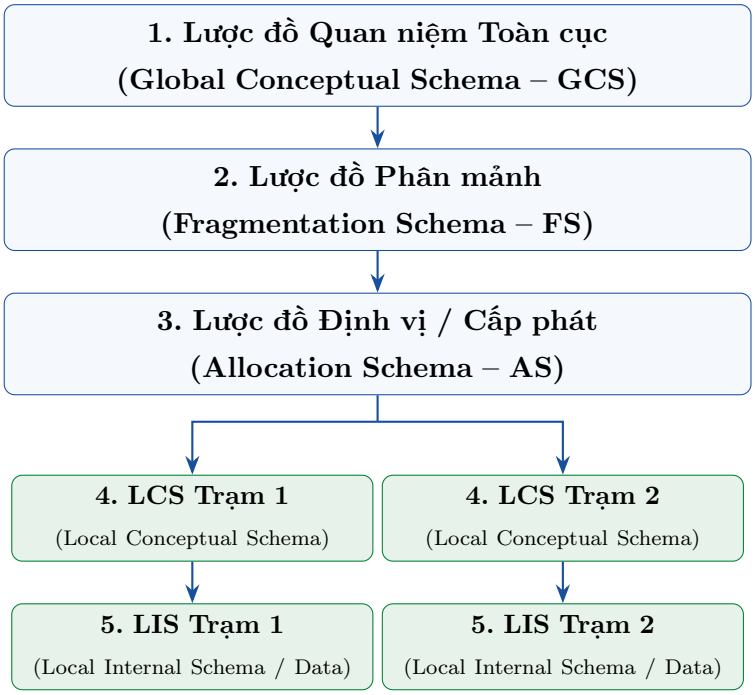
Chương 2

Kiến Trúc Hệ Thống Và Kỹ Thuật Phân Mảnh Dữ Liệu

2.1 Kiến Trúc Tham Chiếu Của Hệ Phân Tán

2.1.1 Kiến trúc phân tầng ANSI/SPARC mở rộng cho DDBMS

Theo nghiên cứu kinh điển của Özsu và Valduriez [1], cấu trúc logic của một DDBMS được chuẩn hóa bằng việc mở rộng kiến trúc ANSI/SPARC truyền thống thành mô hình 5 tầng phân cấp rõ rệt giữa góc nhìn toàn cục và góc nhìn cục bộ:



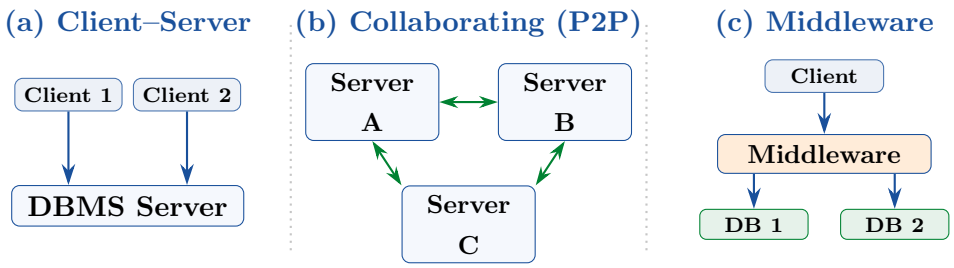
Hình 2.1: Mô hình kiến trúc phân tầng chuẩn của DDBMS (Mở rộng từ ANSI/SPARC)

Hệ thống được chia thành 5 mức trừu tượng chính:

1. **Global Conceptual Schema (GCS):** Mô tả toàn bộ cấu trúc logic của cơ sở dữ liệu phân tán ở mức toàn thể, độc lập hoàn toàn với việc dữ liệu bị cắt nhỏ hay lưu tại nút nào.
2. **Fragmentation Schema (FS):** Xác định ánh xạ 1–N, biểu diễn quy tắc logic mà theo đó mỗi quan hệ toàn cục trong GCS được phân rã thành các phân mảnh nhỏ hơn.
3. **Allocation Schema (AS):** Chỉ định vị trí nút mạng vật lý cụ thể lưu trữ từng phân mảnh cùng số lượng bản sao nhân bản.
4. **Local Conceptual Schema (LCS):** Mô tả cấu trúc logic của phần dữ liệu được giao quản lý tại chính máy chủ cục bộ đó.
5. **Local Internal Schema (LIS):** Tầng vật lý cấp thấp tại từng nút, quản lý trực tiếp cấu trúc chỉ mục, bộ nhớ và tệp tin trên ổ đĩa.

2.1.2 Mô hình Client–Server, Collaborating Server và Middleware

Về mặt triển khai hạ tầng phần mềm và mạng, Hệ quản trị CSDL phân tán được phân loại theo ba mô hình kiến trúc điều phối chính:



Hình 2.2: Ba mô hình kiến trúc hạ tầng điều phối trong DDBMS

Mô hình Client–Server

Phân tách rõ ràng giữa máy khách (*Client*) và máy chủ cơ sở dữ liệu (*Server*):

- **Cơ chế hoạt động:** Phía Client đảm nhận việc xử lý giao diện người dùng, tiếp nhận nghiệp vụ và gửi câu lệnh truy vấn xuống Server. Phía Server thực hiện phân tích cú pháp, tối ưu hóa câu lệnh, truy xuất đĩa cứng và trả về tập kết quả.
- **Đánh giá:** Triển khai đơn giản, dễ bảo trì; tuy nhiên tính mở rộng bị giới hạn bởi năng lực xử lý của máy chủ trung tâm và dễ trở thành điểm nghẽn cổ chai.

Mô hình Collaborating Server (Peer-to-Peer)

Tất cả các máy chủ trong mạng đều có vai trò ngang hàng và tích hợp giao thức truyền thông liên trạm:

- **Cơ chế hoạt động:** Khi nhận một truy vấn từ người dùng, bất kỳ nút mạng nào cũng có thể trở thành **Nút điều phối (*Coordinator*)** cho phiên làm việc đó. Nút này tự động phân tích câu lệnh, liên lạc với các nút ngang hàng để yêu cầu xử lý dữ liệu cục bộ, sau đó tổng hợp các tập kết quả bộ phận để trả về cho người dùng.
- **Đánh giá:** Khả năng chịu lỗi rất cao (không phụ thuộc vào một máy chủ trung tâm duy nhất) và mở rộng ngang linh hoạt; đổi lại, cơ chế quản lý giao dịch và phát hiện bế tắc toàn cục đòi hỏi các giải thuật phân tán tương đối phức tạp.

Mô hình Middleware

Sử dụng một lớp phần mềm trung gian (*Middleware*) làm cầu nối giữa tầng ứng dụng và các hệ quản trị CSDL cục bộ bên dưới:

- **Cơ chế hoạt động:** Tầng Middleware đóng vai trò tiếp nhận truy vấn toàn cục, nắm giữ danh mục vị trí và sử dụng các bộ chuyển đổi giao thức để thông dịch câu lệnh xuống từng hệ quản trị CSDL thành phần (thậm chí khác biệt về hệ điều hành hoặc loại phần mềm DBMS). Bản thân Middleware chỉ làm nhiệm vụ điều phối và chuyển tiếp mà không trực tiếp sở hữu tệp lưu trữ dữ liệu gốc.
- **Đánh giá:** Là giải pháp lý tưởng để xây dựng các hệ cơ sở dữ liệu đa hệ thống và tích hợp các hệ thống kế thừa; tuy nhiên cần thiết lập cơ chế dự phòng cho cụm Middleware để tránh biến nó thành đơn điểm lỗi của hệ thống.

2.1.3 Hệ phân tán đồng nhất, không đồng nhất và Gateway Protocols

Dựa trên mức độ tương thích về phần mềm và mô hình dữ liệu giữa các nút mạng, DDBMS được phân loại thành hai hình thái chính:

Hệ phân tán đồng nhất (Homogeneous DDBMS)

Tất cả các trạm trong mạng đều vận hành cùng một loại phần mềm DBMS (ví dụ: toàn bộ các nút đều sử dụng PostgreSQL hoặc MySQL) và chia sẻ chung một mô hình dữ liệu.

- **Cơ chế hoạt động:** Các trạm sử dụng chung định dạng lưu trữ đĩa, cấu trúc System Catalog và giao thức truyền thông nội bộ. Nhờ đó, việc trao đổi dữ liệu trung gian và điều phối tối ưu hóa truy vấn toàn cục diễn ra trực tiếp mà không cần qua tầng chuyển ngữ.

- **Ưu thế:** Dễ dàng triển khai các giải thuật cam kết phân tán (2PC), đồng bộ bản sao và tối ưu hóa truy vấn toàn diện với chi phí tài nguyên thấp.

Hệ phân tán không đồng nhất (Heterogeneous DDBMS)

Các nút mạng vận hành các hệ quản trị DBMS khác nhau (như trạm dùng Oracle, trạm dùng MySQL) hoặc khác biệt về mô hình dữ liệu (quan hệ kết hợp NoSQL).

- **Thách thức kỹ thuật:** Hệ thống phải đối mặt với sự khác biệt về phương ngữ SQL, xung đột ngữ nghĩa dữ liệu và rào cản về tính tự trị cục bộ.
- **Vai trò của Gateway Protocols:** Để liên kết các trạm không đồng nhất, hệ thống sử dụng các cổng nối (*Gateways*). Gateway đóng vai trò biên dịch câu truy vấn từ ngôn ngữ toàn cục sang cú pháp của DBMS cục bộ, ánh xạ kiểu dữ liệu tương đương và bọc các tiến trình giao dịch cục bộ để đảm bảo an toàn ACID.

Bảng 2.1: So sánh đặc tính giữa Hệ phân tán đồng nhất và không đồng nhất

Tiêu chí	**Hệ phân tán đồng nhất**	**Hệ phân tán không đồng nhất**
Phần mềm DBMS	Cùng một loại DBMS trên toàn mạng.	Đa dạng các DBMS khác nhau tại mỗi trạm.
Tính tự trị cục bộ	Thấp; phối hợp chặt chẽ theo chuẩn chung.	Rất cao; mỗi nút bảo toàn quyền quản trị riêng.
Tối ưu truy vấn	Hiệu quả cao nhờ hiểu rõ thông số các nút.	Khó khăn do thiếu thông tin thống kê chi tiết.
Độ phức tạp tích hợp	Thấp; thuận lợi mở rộng cụm (Scale-out).	Rất cao; đòi hỏi Gateway biên dịch phức tạp.

2.2 Kỹ Thuật Phân Mảnh Dữ Liệu (Data Fragmentation)

Phân mảnh là quá trình phân rã một quan hệ toàn cục R thành các phân mảnh $R_1, R_2, \ldots, R_n$ để phân bố vật lý tại các trạm nhằm tối ưu tính cục bộ tham chiếu và giảm chi phí truyền mạng.

2.2.1 Phân mảnh ngang (Horizontal Fragmentation)

Phân mảnh ngang chia nhỏ bảng theo từng bộ dữ liệu dựa trên phép chọn (σ).

1. Phân mảnh ngang nguyên thủy (PHF – Primary Horizontal Fragmentation): Được định nghĩa thông qua việc sử dụng tập các vị từ đơn giản $Pr = \{p_1, p_2, \ldots, p_k\}$. Để việc phân mảnh tối ưu, tập Pr phải thỏa mãn hai tiêu chuẩn:

- **Tính đầy đủ (Completeness):** Một tập vị từ Pr được gọi là đầy đủ khi và chỉ khi xác suất truy cập vào hai bộ bất kỳ thuộc cùng một phân mảnh là hoàn toàn đồng đều đối với mọi ứng dụng.
- **Tính tối thiểu (Minimality):** Không có bất kỳ vị từ nào trong Pr bị dư thừa.

Từ tập Pr, ta sinh ra tập các **Vị từ hội sơ cấp** (*Minterm Predicates*) $M = \{m_1, m_2, \ldots, m_z\}$ theo công thức:

$$m_i = \bigwedge_{p_j \in Pr} p_j^* \quad \text{với } p_j^* \in \{p_j, \neg p_j\}$$

Mỗi phân mảnh ngang nguyên thủy R_i là kết quả của phép chọn theo một vị từ hội sơ cấp:

$$R_i = \sigma_{m_i}(R)$$

Ví dụ: Phân mảnh ngang nguyên thủy trên quan hệ NhanVien

Xét quan hệ NhanVien(MaNV, HoTen, ChiNhanh, Luong) với 2 ứng dụng truy vấn chính theo trạm làm việc và mức lương.

1. Xác định tập vị từ đơn giản (Pr):

$$Pr = \{p_1 : \text{ChiNhanh} = \text{'HN'}, \quad p_2 : \text{Luong} > 20000000\}$$

2. Sinh các vị từ hội sơ cấp ($m_i = \bigwedge p_j^*$):

- $m_1 = p_1 \wedge p_2$: ChiNhanh = 'HN' $\wedge$ Luong $>$ 20000000
- $m_2 = p_1 \wedge \neg p_2$: ChiNhanh = 'HN' $\wedge$ Luong $\leq$ 20000000
- $m_3 = \neg p_1 \wedge p_2$: ChiNhanh $\neq$ 'HN' $\wedge$ Luong $>$ 20000000
- $m_4 = \neg p_1 \wedge \neg p_2$: ChiNhanh $\neq$ 'HN' $\wedge$ Luong $\leq$ 20000000

3. Cắt dữ liệu thành các phân mảnh ($NV_i = \sigma_{m_i}$(NhanVien)):

- $NV_1 = \sigma_{m_1}(\text{NhanVien})$, $NV_2 = \sigma_{m_2}(\text{NhanVien}) \implies$ Lưu trữ tại **Trạm Hà Nội**.
- $NV_3 = \sigma_{m_3}(\text{NhanVien})$, $NV_4 = \sigma_{m_4}(\text{NhanVien}) \implies$ Lưu trữ tại **Trạm TP.HCM**.

2. Phân mảnh ngang dẫn xuất (DHF – Derived Horizontal Fragmentation): Phân mảnh quan hệ Thành viên (*Member relation*) R dựa trên các phân mảnh của quan hệ Chủ sở hữu (*Owner relation*) S thông qua phép Nửa nối (Semi-join $\ltimes$):

$$R_i = R \ltimes S_i = \pi_{\text{Attrs}(R)}(R \bowtie S_i)$$

Kỹ thuật này đảm bảo các dòng của bảng con sẽ luôn được lưu trữ cùng trạm mạng với dòng cha chứa khóa ngoại liên kết, loại bỏ triệt để chi phí truyền thông liên trạm khi thực hiện phép nối giữa hai bảng.

2.2.2 Phân mảnh dọc (Vertical Fragmentation)

Chia nhỏ bảng theo từng cột dựa trên phép chiếu (π). Mục tiêu là nhóm các thuộc tính có tần suất truy cập đồng thời cao vào cùng một phân mảnh.

- **Cơ sở toán học:** Quá trình phân mảnh dọc sử dụng **Ma trận sử dụng thuộc tính (Attribute Usage Matrix – AUM)** để tính toán **Ma trận tương quan thuộc tính (Attribute Affinity Matrix – AAM)**. Giá trị tương quan giữa hai thuộc tính A_i, A_j được tính bởi:

$$\text{aff}(A_i, A_j) = \sum_{k:q_k \text{ truy cập } A_i,A_j} \sum_{\forall \text{ Trạm } S_l} \text{freq}_l(q_k) \cdot \text{acc}_l(q_k)$$

  Thuật toán **Gom cụm năng lượng liên kết (Bond Energy Algorithm – BEA)** được áp dụng để hoán vị các hàng và cột của ma trận AAM tạo thành ma trận gom cụm (CAAM), từ đó xác định điểm cắt phân đôi tối ưu.

- **Yêu cầu bắt buộc:** Mỗi phân mảnh dọc bắt buộc phải giữ lại thuộc tính **Khóa chính (Primary Key)** hoặc mã định danh bộ (TID).

- **Tái tạo:** Khôi phục bảng gốc bằng phép Nối tự nhiên không mất mát thông tin ($\bowtie$).

Ví dụ: Phân mảnh dọc và phép nối tái tạo Lossless Join

Chia bảng NhanVien(MaNV, HoTen, ChucVu, Luong, SoTaiKhoan) thành 2 phân mảnh dọc:

$$\text{NV_SoYeu} = \pi_{\text{MaNV,HoTen,ChucVu}}(\text{NhanVien})$$
$$\text{NV_TaiChinh} = \pi_{\text{MaNV,Luong,SoTaiKhoan}}(\text{NhanVien})$$

Tái tạo lại quan hệ gốc thông qua phép Nối tự nhiên ($\bowtie$):

$$\text{NhanVien} = \text{NV_SoYeu} \bowtie_{\text{MaNV}} \text{NV_TaiChinh}$$

Lệnh SQL ghép nối khôi phục dữ liệu ban đầu:

```sql
SELECT A.MaNV, A.HoTen, A.ChucVu, B.Luong, B.SoTaiKhoan
FROM NV_SoYeu A
INNER JOIN NV_TaiChinh B ON A.MaNV = B.MaNV;
```

2.2.3 Phân mảnh hỗn hợp (Hybrid Fragmentation)

Kết hợp lồng nhau giữa phân mảnh ngang và phân mảnh dọc (Ngang trước – Dọc sau hoặc Dọc trước – Ngang sau). Tái tạo bằng cách đảo ngược tuần tự các phép nối ($\bowtie$) và phép hợp ($\bigcup$).

2.3 Các Quy Tắc Bắt Buộc Trong Phân Mảnh

Quá trình phân rã một quan hệ R thành $F_R = \{R_1, R_2, \ldots, R_n\}$ bắt buộc phải tuân thủ nghiêm ngặt 3 quy tắc:

2.3.1 Tính đầy đủ (Completeness)

Mọi mục dữ liệu (bộ dữ liệu hoặc thuộc tính) nằm trong quan hệ gốc R phải xuất hiện trong ít nhất một phân mảnh R_i. Đảm bảo không thất lạc dữ liệu sau phân chia.

2.3.2 Tính tái tạo (Reconstruction)

Bắt buộc phải tồn tại một toán tử đại số quan hệ ∇ có khả năng khôi phục lại quan hệ gốc:

$$R = \nabla R_i, \quad \forall R_i \in F_R$$

Trong đó ∇ là phép Hợp ($\bigcup$) với phân mảnh ngang và phép Nối ($\bowtie$) với phân mảnh dọc.

Ví dụ: Bài toán phản ví dụ về vi phạm tính tái tạo

Xét phép phân mảnh dọc bảng NhanVien thành hai mảnh. Nếu mảnh tài chính bỏ quên thuộc tính khóa chính MaNV:

$$\text{NV_TaiChinh} = \pi_{\text{Luong,SoTaiKhoan}}(\text{NhanVien})$$

Khi đó, hệ thống không thể thực hiện phép nối NV_SoYeu $\bowtie$ NV_TaiChinh để khôi phục lại thông tin chính xác của từng nhân viên, dẫn đến vi phạm nghiêm trọng quy tắc **Reconstruction**. Phép nối không có khóa chung sẽ suy biến thành tích Đề-các, làm sai lệch toàn bộ bộ dữ liệu ban đầu.

2.3.3 Tính tách biệt (Disjointness)

Ngăn chặn hiện tượng dư thừa trùng lặp dữ liệu giữa các phân mảnh:

- **Đối với phân mảnh ngang:** $R_j \cap R_k = \emptyset, \quad \forall j \neq k.$

- **Đối với phân mảnh dọc:** Tính tách biệt áp dụng chặt chẽ trên **các thuộc tính không phải khóa chính**.

2.4 Chiến Lược Phân Bổ và Cấp Phát Dữ Liệu

Sau khi hoàn tất quá trình phân mảnh, bước tiếp theo trong quy trình thiết kế CSDL phân tán là **Cấp phát dữ liệu (Data Allocation)**. Đây là quá trình xác định vị trí lưu trữ vật lý của từng phân mảnh tại các trạm mạng nhằm tối ưu hóa hiệu năng tổng thể của hệ thống.

2.4.1 Bài toán cấp phát dữ liệu

Về mặt toán học, bài toán cấp phát được mô hình hóa như sau:

- **Đầu vào:** Tập các phân mảnh $F = \{F_1, F_2, \ldots, F_n\}$; tập các trạm mạng $S = \{S_1, S_2, \ldots, S_m\}$; ma trận tần suất đọc/ghi của các ứng dụng và chi phí truyền thông giữa các cặp trạm.
- **Biến quyết định:** Ma trận nhị phân $X = [x_{ij}]_{n \times m}$, trong đó:

$$x_{ij} = \begin{cases} 1 & \text{nếu phân mảnh } F_i \text{ được cấp phát tại trạm } S_j \\ 0 & \text{ngược lại} \end{cases}$$

- **Mục tiêu:** Cực tiểu hóa hàm tổng chi phí toàn cục:

$$\min \; C = C_{\text{đọc}} + C_{\text{ghi}} + C_{\text{truyền thông}} + C_{\text{lưu trữ}}$$

  thỏa mãn các ràng buộc về dung lượng đĩa tại mỗi trạm ($\sum_i x_{ij} \cdot \text{size}(F_i) \leq \text{Capacity}_j$) và giới hạn thời gian đáp ứng của truy vấn.

Bài toán cấp phát tổng quát đã được chứng minh là bài toán **NP-đầy đủ (NP-complete)**. Do đó, các bộ tối ưu DDBMS trong thực tế thường áp dụng các giải thuật xấp xỉ (*Heuristics*) để tìm phương án cấp phát gần tối ưu.

2.4.2 Các mô hình cấp phát dữ liệu

Dựa trên giá trị của ma trận cấp phát X, hệ thống phân tán được phân loại thành ba mô hình chính:

Mô hình phân bổ không nhân bản (Non-replicated Allocation)

Mỗi phân mảnh F_i chỉ tồn tại duy nhất tại đúng một trạm mạng trong toàn hệ thống, tức là:

$$\forall i \in \{1, \ldots, n\} : \quad \sum_{j=1}^{m} x_{ij} = 1$$

- **Ưu điểm:** Cực tiểu hóa chi phí lưu trữ; thao tác ghi/cập nhật dữ liệu rất đơn giản và diễn ra tức thì vì không cần cơ chế đồng bộ bản sao liên trạm.
- **Nhược điểm:** Độ sẵn sàng (Availability) và độ chịu lỗi thấp. Nếu trạm chứa phân mảnh gặp sự cố (sập nút), toàn bộ các truy vấn liên quan đến mảnh đó sẽ bị gián đoạn.

Mô hình nhân bản toàn phần (Fully Replicated Allocation)

Mọi phân mảnh đều được sao chép nguyên vẹn tại tất cả các trạm mạng trong hệ thống:

$$\forall i \in \{1, \ldots, n\},\ \forall j \in \{1, \ldots, m\} : \quad x_{ij} = 1$$

- **Ưu điểm:** Tối ưu hóa tối đa thời gian đáp ứng của các truy vấn đọc, do 100% thao tác đọc được phục vụ cục bộ ngay tại trạm phát sinh; đồng thời tận dụng các bản sao để cân bằng tải đọc trên toàn hệ thống; độ chịu lỗi cao.
- **Nhược điểm:** Tiêu tốn dung lượng đĩa gấp m lần; chi phí cập nhật cực kỳ tốn kém do phải thực hiện các giao thức cam kết đồng bộ trên toàn mạng để bảo đảm tính nhất quán dữ liệu.

Mô hình nhân bản từng phần (Partially Replicated Allocation)

Chỉ một số phân mảnh tần suất đọc vượt trội mới được nhân bản tại một số trạm nhất định:

$$\exists i : \quad 1 < \sum_{j=1}^{m} x_{ij} < m$$

Đây là mô hình dung hòa, mang lại hiệu năng thực tế cao nhất nhờ việc cân bằng giữa chi phí truyền thông mạng và chi phí đồng bộ dữ liệu.

Chương 3

Quy Trình Xử Lý Và Tối Ưu Hóa Truy Vấn Phân Tán

3.1 Tổng Quan Về Kiến Trúc Xử Lý Truy Vấn Phân Tán

Trong hệ quản trị cơ sở dữ liệu phân tán (DDBMS), dữ liệu được lưu trữ phân tán trên nhiều trạm (sites hoặc nodes) kết nối qua mạng máy tính. Sự thay đổi về mặt kiến trúc phần cứng này kéo theo sự thay đổi căn bản trong cơ chế biên dịch, tối ưu hóa và thực thi câu lệnh truy vấn.

Khác với hệ thống tập trung nơi chi phí thực thi chủ yếu bị chi phối bởi thao tác I/O đọc/ghi đĩa từ hoặc ổ cứng thể rắn (SSD) và chu kỳ tính toán của CPU, trong môi trường phân tán, **chi phí truyền thông qua mạng (Network Communication Cost)** đóng vai trò là điểm nghẽn lớn nhất (bottleneck) do băng thông hữu hạn và độ trễ đường truyền cao.

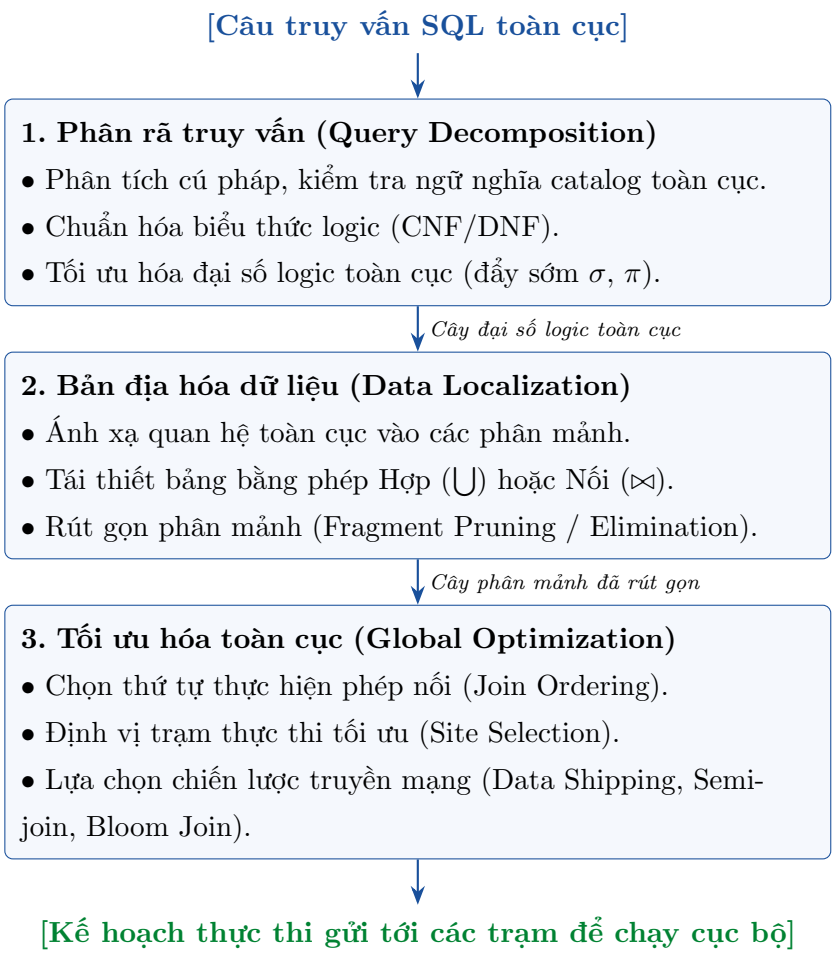
Để biến đổi một câu lệnh truy vấn khai báo cấp cao (như SQL) từ người dùng thành các thao tác vật lý tối ưu trên mạng lưới phân tán, kiến trúc xử lý truy vấn phân tán tiêu chuẩn được tổ chức thành quy trình phân tầng gồm 3 giai đoạn trọng tâm:

3.2 Chi Tiết Các Bước Xử Lý Câu Lệnh Truy Vấn

3.2.1 Phân rã truy vấn (Query Decomposition)

Khái niệm và Mục tiêu

Phân rã truy vấn là bước tiếp nhận câu truy vấn SQL viết trên **lược đồ quan hệ toàn cục (Global Relational Schema)**. Ở bước này, hệ thống **trừu tượng hóa hoàn toàn**



Hình 3.1: Quy trình 3 giai đoạn xử lý và tối ưu hóa truy vấn phân tán

môi trường phân tán: không quan tâm đến việc dữ liệu được phân mảnh thế nào, nhân bản ở đâu hay lưu trữ tại site vật lý nào.

Mục tiêu chính là dịch câu truy vấn khai báo sang dạng biểu diễn hình thức nội bộ – một **Cây đại số quan hệ toàn cục (Global Relational Algebra Tree)** – đồng thời áp dụng các phép biến đổi đại số tương đương để đơn giản hóa cây truy vấn.

Các tiểu bước trong Phân rã truy vấn

Quá trình phân rã truy vấn được chuẩn hóa qua 4 giai đoạn kỹ thuật:

1. **Phân tích cú pháp và kiểm tra ngữ nghĩa (Parsing and Semantic Analysis):**
   - *Kiểm tra cú pháp:* Đảm bảo câu lệnh tuân thủ ngữ pháp chuẩn của SQL.
   - *Kiểm tra từ điển dữ liệu (Catalog Lookup):* Xác thực tên bảng, tên thuộc tính xem có tồn tại trong lược đồ toàn cục hay không; kiểm tra tương thích kiểu dữ liệu.
   - *Kiểm tra quyền truy cập (Authorization):* Xác minh người dùng có đủ đặc quyền

(`SELECT`, `UPDATE`,...) trên các quan hệ được tham chiếu.

2. **Chuẩn hóa biểu thức điều kiện (Normalization):** Mệnh đề `WHERE` được đưa về các dạng chuẩn đại số logic:

   - **Dạng chuẩn hội (Conjunctive Normal Form – CNF):** Là chuỗi các mệnh đề tuyển nối với nhau bằng phép AND:

     $$P = (p_{11} \vee p_{12} \vee \dots) \wedge (p_{21} \vee p_{22} \vee \dots) \wedge \dots$$

     Dạng này đặc biệt thuận tiện cho việc thực hiện quy tắc phân rã phép chọn (Cascade of Selections).

   - **Dạng chuẩn tuyển (Disjunctive Normal Form – DNF):** Là chuỗi các mệnh đề hội nối với nhau bằng phép OR:

     $$P = (p_{11} \wedge p_{12} \wedge \dots) \vee (p_{21} \wedge p_{22} \wedge \dots) \vee \dots$$

3. **Phân tích ngữ nghĩa và loại bỏ dư thừa (Semantic Analysis & Contradiction Elimination):** Hệ thống kiểm tra tính khả thi của các vị từ. Nếu một nhánh vị từ tự mâu thuẫn (như $A > 100 \wedge A < 50$), nhánh đó được đánh giá là False và bị loại bỏ ngay từ cây logic. Đồng thời áp dụng tính chất bắc cầu: nếu $A = B$ và $B = 10$, hệ thống suy diễn thêm vị từ $A = 10$.

4. **Tối ưu hóa đại số logic toàn cục (Algebraic Optimization):** Áp dụng các quy tắc tương đương đại số quan hệ độc lập phân tán nhằm giảm kích thước dữ liệu trung gian:

   - **Đẩy sớm phép chọn (Push Down Selections):** $\sigma_\theta(R \bowtie S) \equiv (\sigma_\theta(R)) \bowtie S$ nếu điều kiện θ chỉ liên quan đến quan hệ R, giúp giảm nhanh số lượng bộ trước khi thực hiện phép nối đắt đỏ.
   - **Đẩy sớm phép chiếu (Push Down Projections):** $\Pi_L(R \bowtie S) \equiv \Pi_L(\Pi_{L_1}(R) \bowtie \Pi_{L_2}(S))$, loại bỏ sớm các cột không cần thiết để giảm độ rộng dòng dữ liệu.
   - **Nhóm các toán tử đơn nguyên:** Gom các chuỗi phép chọn và chiếu liên tiếp để thực thi trong cùng một lần duyệt dữ liệu.

3.2.2 Bản địa hóa dữ liệu (Data Localization)

Khái niệm và Bối cảnh

Sau khi có cây đại số quan hệ toàn cục, hệ thống bước vào giai đoạn Bản địa hóa dữ liệu. Tại đây, tính trong suốt phân mảnh (Fragmentation Transparency) được mở khóa:

hệ thống bắt đầu xem xét cấu trúc phân mảnh và vị trí vật lý của dữ liệu. Nhiệm vụ của bản địa hóa là thay thế mỗi quan hệ toàn cục ở các nút lá bằng **chương trình tái thiết (Reconstruction Program)** tương ứng từ các phân mảnh cục bộ, sau đó thực hiện **cắt tỉa các phân mảnh không liên quan (Fragment Pruning / Elimination)**.

Các dạng phân mảnh và Quy tắc tái thiết

1. **Phân mảnh ngang (Horizontal Fragmentation):** $R = \bigcup_{i=1}^{n} R_i = R_1 \cup R_2 \cup \cdots \cup R_n$.
2. **Phân mảnh dọc (Vertical Fragmentation):** $R = \bowtie_K R_j = R_1 \bowtie_K R_2 \bowtie_K \ldots \bowtie_K R_m$ (với K là khóa chính).
3. **Phân mảnh hỗn hợp (Hybrid Fragmentation):** Kết hợp đan xen giữa phép Hợp ($\bigcup$) và phép Nối ($\bowtie$).

Cơ chế Rút gọn phân mảnh (Fragment Pruning)

Kỹ thuật Rút gọn phân mảnh sử dụng đại số quan hệ và logic toán học để triệt tiêu các nhánh rỗng:

- **Phân phối phép chọn qua phép hợp:** $\sigma_C(R_1 \cup R_2) \equiv \sigma_C(R_1) \cup \sigma_C(R_2)$.
- **Kiểm tra mâu thuẫn vị từ (Predicate Contradiction Check):** Mỗi phân mảnh R_i gắn liền với vị từ phân mảnh ϕ_i. Biểu thức truy vấn trên mảnh trở thành $\sigma_C(R_i) \equiv \sigma_{C \wedge \phi_i}(R)$. Nếu $(C \wedge \phi_i) \equiv$ False, phân mảnh R_i chắc chắn không chứa bộ nào thỏa mãn: $\sigma_{C \wedge \phi_i}(R) = \emptyset$. Hệ thống cắt tỉa toàn bộ nhánh R_i khỏi cây truy vấn.
- **Phân phối phép chọn và chiếu qua phép nối:** Nếu điều kiện chọn C và danh sách chiếu chỉ nằm hoàn toàn trong một phân mảnh dọc R_1, các phân mảnh dọc khác không cần tham gia vào phép nối tái thiết.

3.2.3 Tối ưu hóa toàn cục (Global Optimization)

Khái niệm và Mục tiêu

Tối ưu hóa toàn cục là bước quyết định kế hoạch vật lý để thực thi cây truy vấn phân mảnh đã rút gọn, trả lời bài toán: *"Thao tác nào được thực hiện trước, thuật toán nào được áp dụng, và phép toán đó diễn ra tại trạm nào?"* Mục tiêu là tìm kế hoạch thực thi phân tán sao cho tổng tài nguyên tiêu thụ hoặc thời gian đáp ứng (*Response Time*) là nhỏ nhất.

Mô hình ước lượng chi phí phân tán

Hàm mục tiêu chi phí trong DDBMS bao gồm 3 thành phần chính:

$$\text{Total Cost} = \text{Cost}_{\text{CPU}} + \text{Cost}_{\text{I/O}} + \text{Cost}_{\text{COM}}$$

Trong đó:

- Cost_{CPU}: Chi phí tính toán CPU cho so sánh bộ, băm, sắp xếp.
- $\text{Cost}_{\text{I/O}}$: Đo bằng số khối đĩa truy xuất ($b \times t_T$) cộng số lần tìm kiếm ngẫu nhiên ($S \times t_S$).
- Cost_{COM}: Chi phí truyền thông mạng, đóng vai trò chi phối trên mạng WAN:

$$\text{Cost}_{\text{COM}} = \text{Message Delay (Độ trễ khởi tạo)} + \frac{\text{Data Size (Kích thước dữ liệu)}}{\text{Bandwidth (Băng thông)}}$$

Không gian tìm kiếm và các quyết định tối ưu

- **Xác định thứ tự thực hiện phép nối (Join Ordering):** Áp dụng quy hoạch động hoặc Heuristic để hạn chế sinh kích thước dữ liệu trung gian lớn.
- **Định vị toán tử và trạm thực thi (Site Selection):** Đặt toán tử nối tại Site 1, Site 2 hoặc Site trung gian tùy thuộc vào tương quan kích thước sau khi đã lọc cục bộ.
- **Các chiến lược nối phân tán:**
  - *Truyền toàn bộ (Data Shipping):* Gửi nguyên vẹn quan hệ nhỏ hơn sang trạm chứa quan hệ lớn hơn.
  - *Kỹ thuật Nửa nối (Semi-join Optimization):* Định nghĩa $R \ltimes_A S = \Pi_R(R \bowtie_A S)$. Sử dụng phép chiếu lấy tập khóa nối $Temp = \Pi_A(S)$, gửi sang trạm chứa R để lọc lấy $R' = R \ltimes S$, rồi mới gửi R' về trạm của S. Điều kiện có lợi: $\text{Size}(\Pi_A(S)) + \text{Size}(R \ltimes S) < \text{Size}(R)$.
  - *Bloom Join:* Sử dụng mảng bit băm (Bloom Filter) kích thước vài KB để lọc nhanh dữ liệu trước khi truyền qua mạng.

3.3 Bài Toán Định Lượng Minh Họa Phép Toán Semi-Join

Để minh chứng tường minh cách phép toán Semi-join giảm thiểu dung lượng đường truyền mạng, xét một bài toán định lượng cụ thể:

3.3.1 Bối cảnh và Thông số Kỹ thuật

Hệ thống CSDL phân tán gồm 2 trạm kết nối qua mạng WAN:

- **Site 1 (Hà Nội):** Lưu trữ bảng **Customer** (R) chứa thông tin khách hàng:
  - Số lượng bản ghi: 100.000 dòng.
  - Kích thước một bản ghi: $l_R = 200$ bytes (gồm `CustomerID` [4 bytes], `Name`, `Address`,...).
  - Tổng kích thước bảng R: 100.000 × 200 bytes = 20.000.000 bytes = 20 MB = 20.000 KB.
  - Thuộc tính nối: `CustomerID` (4 bytes).
- **Site 2 (TP.HCM):** Lưu trữ bảng **VIP_Orders** (S) chứa đơn hàng giá trị cao:
  - Số lượng bản ghi: 1.000 dòng.
  - Kích thước một bản ghi: $l_S = 50$ bytes.
  - Thuộc tính nối: `CustomerID` (4 bytes).
  - *Đặc điểm tập dữ liệu:* 1.000 đơn hàng VIP này chỉ liên quan đến **500 khách hàng duy nhất** ($\Pi_{\text{CustomerID}}(S)$ chứa đúng 500 giá trị ID).
- **Yêu cầu truy vấn:** Thực hiện phép nối $R \bowtie_{\text{CustomerID}} S$ và trả kết quả tại **Site 2**.

3.3.2 Phân tích So sánh 2 Chiến lược Thực thi

Chiến lược 1: Truyền dữ liệu trực tiếp (Direct Relation Shipping): Site 1 đóng gói và truyền toàn bộ bảng Customer (R) qua mạng sang Site 2 để nối cục bộ:

$$\text{Dung lượng truyền} = 100.000 \text{ dòng} \times 200 \text{ bytes} = \mathbf{20.000} \text{ KB (20 MB)}$$

Chiến lược 2: Kỹ thuật giảm tải bằng Semi-join ($R \ltimes S$):

1. **Bước 1 (Site 2 → Site 1):** Site 2 trích xuất danh sách các mã CustomerID duy nhất: Temp = $\Pi_{\text{CustomerID}}(S)$ và gửi sang Site 1:

$$\text{Dung lượng B1} = 500 \text{ mã ID} \times 4 \text{ bytes} = 2.000 \text{ bytes} = \mathbf{2} \text{ KB}$$

2. **Bước 2 (Tại Site 1):** Site 1 thực hiện Semi-join: $R' = R \ltimes S = R \bowtie$ Temp, thu được đúng 500 dòng khách hàng đầy đủ.
3. **Bước 3 (Site 1 → Site 2):** Site 1 gửi tập R' đã thu gọn sang Site 2:

$$\text{Dung lượng B3} = 500 \text{ dòng} \times 200 \text{ bytes} = 100.000 \text{ bytes} = \mathbf{100} \text{ KB}$$

4. **Bước 4 (Tại Site 2):** Site 2 tiến hành nối $R' \bowtie S$ để hoàn tất kết quả.

Tổng dung lượng dữ liệu truyền qua mạng của Semi-join:

Tổng dung lượng truyền = Dung lượng B1+Dung lượng B3 = 2 KB+100 KB = **102** KB

3.3.3 Bảng Tổng hợp và Đánh giá Hiệu quả Định lượng

Bảng 3.1: So sánh định lượng giữa Truyền trực tiếp và Kỹ thuật Semi-join

Tiêu chí đánh giá	**Chiến lược 1 (Truyền trực tiếp)**	**Chiến lược 2 (Sử dụng Semi-join)**	**Mức độ tối ưu / Đánh đổi**
Số lượt truyền mạng	1 lượt (Site 1 → Site 2)	2 lượt (Site 2 → Site 1 → Site 2)	Tăng 1 lượt bắt tay thông điệp (Round-trip)
Dung lượng mạng B1	0 KB	2 KB	Không đáng kể
Dung lượng truyền chính	20.000 KB	100 KB	Giảm 200 lần
Tổng dung lượng mạng	**20.000 KB (20 MB)**	**102 KB**	**Tiết kiệm 19.898 KB** ($\approx 99,5\%$)
Thời gian truyền (Băng thông 10 Mbps)	$\approx$ 16 giây	$\approx 0,08$ giây	Nhanh hơn gần 200 lần

3.3.4 Nguyên tắc Áp dụng và Trade-off Kỹ thuật

- **Hiệu quả cốt lõi:** Semi-join đạt mức tiết kiệm tới $99,5\%$ dung lượng đường truyền nhờ loại bỏ 99.500 bản ghi không tham gia vào kết quả nối ngay tại nguồn.
- **Điều kiện áp dụng tối ưu:** Phát huy tác dụng tối đa khi **độ chọn lọc (Selectivity)** của phép nối thấp (chỉ tỷ lệ nhỏ các bộ của bảng lớn thỏa mãn điều kiện nối).
- **Đánh đổi kỹ thuật (Trade-off):** Semi-join phát sinh thêm 1 lượt bắt tay mạng (Latency Round-trip) và chi phí xử lý phép chiếu/nối trung gian. Bộ tối ưu hóa chỉ chọn Semi-join khi chi phí dung lượng tiết kiệm được vượt trội so với độ trễ phát sinh do thông điệp bổ sung.

3.4 Phân Tích Tình Huống Xuyên Suốt (Case Study Truy Vấn Đa Trạm)

3.4.1 Bối cảnh Hệ thống

Xét hệ thống quản lý bán hàng của một chuỗi bán lẻ gồm hai quan hệ toàn cục:

- `Customer(CustomerID, Name, City, Balance)`
- `Orders(OrderID, CustomerID, OrderDate, TotalAmount)`

Cấu hình phân tán:

- Quan hệ `Customer` được **phân mảnh ngang** theo thành phố:
  - $C_1 = \sigma_{\text{City = 'Hanoi'}}(Customer)$ lưu tại **Site 1 (Hà Nội)** (5.000 bộ, 100 B/bộ → 500 KB).
  - $C_2 = \sigma_{\text{City = 'Saigon'}}(Customer)$ lưu tại **Site 2 (Sài Gòn)** (10.000 bộ, 100 B/bộ → 1 MB).
  - $C_3 = \sigma_{\text{City = 'Danang'}}(Customer)$ lưu tại **Site 3 (Đà Nẵng)** (2.000 bộ, 100 B/bộ → 200 KB).
- Quan hệ `Orders` không phân mảnh, đặt tập trung tại **Site 4 (Data Center – Huế)** với 1.000.000 đơn hàng (50 B/bộ → 50 MB).

Truy vấn của người dùng phát xuất từ **Site 1**:

```
SELECT C.Name, O.OrderID, O.TotalAmount
FROM Customer C JOIN Orders O ON C.CustomerID = O.CustomerID
WHERE C.City = 'Hanoi' AND O.TotalAmount > 10000000;
```

3.4.2 Diễn tiến qua 3 Bước Xử lý

Giai đoạn 1: Phân rã truy vấn (Query Decomposition): Áp dụng quy tắc tương đương đẩy phép chọn và chiếu xuống trước phép nối:

- Lọc Customer với $\sigma_{\text{City = 'Hanoi'}}$; Lọc Orders với $\sigma_{\text{TotalAmount} > 10000000}$.
- Chiếu các cột cần thiết: trên Customer lấy $\Pi_{\text{CustomerID, Name}}$, trên Orders lấy $\Pi_{\text{CustomerID, OrderID, Tota}}$

Biểu thức đại số quan hệ logic toàn cục:

$$T_1 = \Pi_{\text{Name, OrderID, TotalAmount}}\Big(\Pi_{\text{CustomerID, Name}}(\sigma_{\text{City = 'Hanoi'}}(Customer)) \bowtie_{\text{CustomerID}} \Pi_{\text{CustomerID, OrderID, TotalAmount}}(\sigma_{\text{TotalAmount} > 10\text{M}}(Orders))\Big)$$

Giai đoạn 2: Bản địa hóa dữ liệu (Data Localization):

1. **Thay thế quan hệ toàn cục bằng biểu thức tái thiết:** $Customer = C_1 \cup C_2 \cup C_3$. Biểu thức trở thành:

$$\sigma_{\text{City = 'Hanoi'}}(C_1 \cup C_2 \cup C_3) \equiv \sigma_{\text{City = 'Hanoi'}}(C_1) \cup \sigma_{\text{City = 'Hanoi'}}(C_2) \cup \sigma_{\text{City = 'Hanoi'}}(C_3)$$

2. **Rút gọn phân mảnh (Fragment Pruning):**
   - Với C_1: (City = 'Hanoi') $\wedge$ (City = 'Hanoi') $\equiv$ True $\implies$ Giữ lại nhánh C_1.
   - Với C_2: (City = 'Saigon') $\wedge$ (City = 'Hanoi') $\equiv$ False $\implies \emptyset$ (Cắt tỉa).
   - Với C_3: (City = 'Danang') $\wedge$ (City = 'Hanoi') $\equiv$ False $\implies \emptyset$ (Cắt tỉa).

Cây truy vấn phân mảnh sau rút gọn:

$$T_2 = \Pi_{\text{Name, OrderID, TotalAmount}}\Big(\Pi_{\text{CustomerID, Name}}(C_1) \bowtie_{\text{CustomerID}} \Pi_{\text{CustomerID, OrderID, TotalAmount}}(\sigma_{\text{TotalAmount} > 10\text{M}}(Orders))\Big)$$

Hệ thống hoàn toàn loại trừ Site 2 và Site 3 ra khỏi kế hoạch thực thi.

Giai đoạn 3: Tối ưu hóa toàn cục (Global Optimization): Dữ liệu cần xử lý:

- C_1 tại **Site 1 (Hà Nội)**: 5.000 bộ, chiếu lấy (CustomerID, Name) (30 B/bộ), kích thước Size(C_1') = 150 KB.
- Orders tại **Site 4 (Huế)**: Sau khi lọc $\sigma_{\text{TotalAmount} > 10\text{M}}$, còn 20.000 đơn hàng, chiếu lấy (CustomerID, OrderID, TotalAmount) (20 B/bộ), kích thước Size(O') = 400 KB.

Bộ tối ưu hóa áp dụng Semi-join ($C_1' \ltimes O'$):

1. Site 1 gửi danh sách CustomerID từ C_1': 5.000 ID $\times$ 4 bytes = 20 KB sang Site 4.
2. Site 4 dùng 20 KB này lọc O', chỉ còn lại 3.000 đơn hàng thuộc về khách hàng Hà Nội.
3. Site 4 gửi 3.000 đơn hàng này về Site 1: 3.000 $\times$ 20 bytes = 60 KB.
4. Site 1 thực hiện nối cục bộ tạo kết quả cuối cùng.

Tổng dữ liệu truyền qua mạng: 20 KB + 60 KB = **80** KB (tiết kiệm 80% dung lượng so với 400 KB khi truyền trực tiếp).

3.5 Tổng Kết Và So Sánh Các Giai Đoạn Xử Lý Truy Vấn

Ba giai đoạn **Phân rã truy vấn**, **Bản địa hóa dữ liệu** và **Tối ưu hóa toàn cục** tạo thành chu trình chuyển đổi hoàn chỉnh trong DDBMS: đảm bảo câu lệnh dịch chính xác,

Bảng 3.2: Tổng hợp đặc tính kỹ thuật của 3 giai đoạn xử lý truy vấn phân tán

Tiêu chí	**2.1 Phân rã truy vấn**	**2.2 Bản địa hóa dữ liệu**	**2.3 Tối ưu hóa toàn cục**
Mức độ trừu tượng	Độc lập hoàn toàn với phân tán và phân mảnh.	Phụ thuộc phân mảnh, độc lập với vị trí trạm cụ thể.	Gắn liền với cấu trúc vật lý mạng và vị trí trạm.
Đầu vào	Câu truy vấn SQL toàn cục.	Cây đại số quan hệ logic toàn cục + Lược đồ phân mảnh.	Cây truy vấn phân mảnh đã rút gọn + Thống kê mạng.
Thao tác then chốt	Parsing, chuẩn hóa CNF/DNF, đẩy phép chọn/chiếu xuống sớm.	Tái thiết bảng qua Hợp/Nối, kiểm tra mâu thuẫn vị từ, cắt tỉa phân mảnh.	Ước lượng chi phí, chọn thứ tự join, chọn site, chọn Semi-join/Bloom-join.
Đầu ra	Cây đại số quan hệ logic toàn cục.	Cây truy vấn phân mảnh đã rút gọn (Fragment Tree).	Kế hoạch thực thi phân tán tối ưu (Execution Plan).
Trọng tâm tối ưu	Giảm số lượng bộ và thuộc tính trung gian logic.	Loại bỏ các phân mảnh rỗng không đóng góp kết quả.	Giảm thiểu băng thông truyền mạng và thời gian đáp ứng.

loại trừ các vùng dữ liệu không liên quan và đưa ra kế hoạch thực thi tối thiểu hóa chi phí truyền thông mạng.

Chương 4

Quản Lý Giao Dịch Và Kiểm Soát Đồng Thời Phân Tán

4.1 Tổng Quan Giao Dịch Và Kiến Trúc Xử Lý Phân Tán

4.1.1 Giao dịch phân tán là gì?

Giao dịch (transaction) là một đơn vị công việc logic gồm nhiều thao tác đọc/ghi dữ liệu, cần được thực hiện theo một kết quả cuối cùng thống nhất: hoặc được công nhận hoàn toàn, hoặc bị hủy bỏ trọn vẹn. Trong hệ phân tán, những thao tác của cùng một giao dịch có thể được thực thi ở nhiều nút lưu trữ (site/node) khác nhau. Vì không có một bộ nhớ và một bộ quản lý lỗi dùng chung, việc quyết định giao dịch thành công bắt buộc phải thông qua cơ chế trao đổi thông điệp điều phối giữa các nút.

4.1.2 Các thành phần chức năng

4.1.3 Phân biệt giao dịch, cam kết và sao chép dữ liệu

Giao dịch phân tán có thể truy cập các phân mảnh khác nhau; sao chép (replication) là việc duy trì nhiều bản sao của cùng một dữ liệu. Hai vấn đề có thể cùng xuất hiện nhưng không đồng nhất: Giao thức cam kết nguyên tử như 2PC quyết định một giao dịch đa nút commit hay abort; trong khi đó, một giao thức đồng thuận hoặc cơ chế sao chép mới xác định cách các bản sao thống nhất thứ tự cập nhật. Do đó, việc sử dụng 2PC không tự động bảo đảm mọi lần đọc trên mọi bản sao đều thấy dữ liệu mới nhất.

Bảng 4.1: Các thành phần chức năng trong kiến trúc xử lý giao dịch phân tán

Thành phần	**Vai trò chính**
Client / Ứng dụng	Khởi tạo giao dịch T và phát yêu cầu `COMMIT` hoặc `ROLLBACK`.
Transaction Coordinator	Điều phối các nút tham gia; tập hợp phiếu bầu, đưa ra quyết định cam kết toàn cục.
Participant (các trạm)	Chạy phần giao dịch cục bộ, khóa dữ liệu, ghi log và thực thi quyết định cuối cùng.
Concurrency Controller	Sắp lịch giao dịch bằng cơ chế khóa hoặc nhãn thời gian để ngăn ngừa xung đột.
Log & Recovery Manager	Ghi nhật ký lên bộ nhớ bền vững (WAL), thực hiện REDO/UNDO để phục hồi sau sự cố.
Mạng truyền thông	Truyền thông điệp PREPARE, phiếu bầu, COMMIT/ABORT và ACK; chịu độ trễ hoặc lỗi mạng.

4.2 Thuộc Tính ACID Trên Môi Trường Nhiều Máy Chủ

Bảng 4.2: Bảo đảm 4 thuộc tính ACID trong môi trường phân tán

Thuộc tính	**Ý nghĩa trong hệ phân tán**	**Cơ chế hỗ trợ**
Atomicity – Nguyên tử	Không xuất hiện tình trạng một phần trạm commit, phần trạm khác abort.	Giao thức 2PC/3PC và log khôi phục, điều phối quyết định toàn cục.
Consistency – Nhất quán	Nếu trạng thái trước hợp lệ và giao dịch thỏa mãn ràng buộc, trạng thái sau vẫn hợp lệ.	Ràng buộc dữ liệu, logic nghiệp vụ, kiểm soát thực thi và cam kết nguyên tử.
Isolation – Cô lập	Các giao dịch đồng thời không làm lộ kết quả trung gian; mức cao nhất là khả tuần tự.	Distributed 2PL, Timestamp Ordering, thuật toán điều khiển đồng thời.
Durability – Bền vững	Giao dịch đã commit phải phục hồi trọn vẹn sau sự cố trên các nút.	Write-Ahead Logging (WAL) trên lưu trữ bền vững và cơ chế sao chép.

Nhận xét quan trọng: 2PC giải quyết chủ yếu bài toán thống nhất quyết định COMMIT/ABORT (Atomicity), không tự nó giải quyết Isolation hay bảo đảm toàn vẹn ràng buộc nghiệp vụ. Một thiết kế ACID đầy đủ cần phối hợp lịch giao dịch hợp lệ và cơ chế lưu trữ/phục hồi tin

cậy. Đặc biệt, không nên nhầm lẫn khái niệm Consistency trong ACID (bảo toàn bất biến toàn vẹn dữ liệu) với Consistency trong định lý CAP (tương đương với Linearizability).

4.3 Giao Thức Cam Kết Hai Pha (Two-Phase Commit – 2PC)

4.3.1 Mục đích và giả định

2PC là giao thức thống nhất quyết định cam kết nguyên tử giữa một Coordinator và n Participant. Mô hình cơ bản giả định các nút có nhật ký bền vững, có khả năng phục hồi sau sự cố sập, thông điệp có thể gửi lại, giao dịch định danh duy nhất và không có lỗi kiểu Byzantine (nút không gian lận). Cơ chế thời gian chờ (*timeout*) hỗ trợ nhận biết bất thường nhưng không thể phân biệt tuyệt đối giữa máy sập và mạng trễ/chia cắt.

4.3.2 Pha 1 – PREPARE / VOTING (Chuẩn bị và Bỏ phiếu)

1. Ứng dụng hoàn tất công việc logic và yêu cầu Coordinator cam kết giao dịch T.
2. Coordinator gửi thông điệp `PREPARE(T)` đến từng Participant tham gia.
3. Mỗi Participant xác minh khả năng commit cục bộ (không vi phạm ràng buộc, đã ghi log cần thiết).
4. Nếu sẵn sàng, Participant ghi bền vững trạng thái `PREPARED(T)`, giữ các khóa tài nguyên và trả lời `VOTE-YES`.
5. Nếu không sẵn sàng, Participant gửi `VOTE-NO` và tự hủy phần giao dịch cục bộ.

Điểm then chốt: Sau khi đã gửi `VOTE-YES`, Participant không còn quyền tự ý `ABORT`. Nó bắt buộc phải chấp hành quyết định cuối cùng từ Coordinator. Đây là nguồn gốc của trạng thái bế tắc chờ (*In-doubt*) khi mất liên lạc mạng.

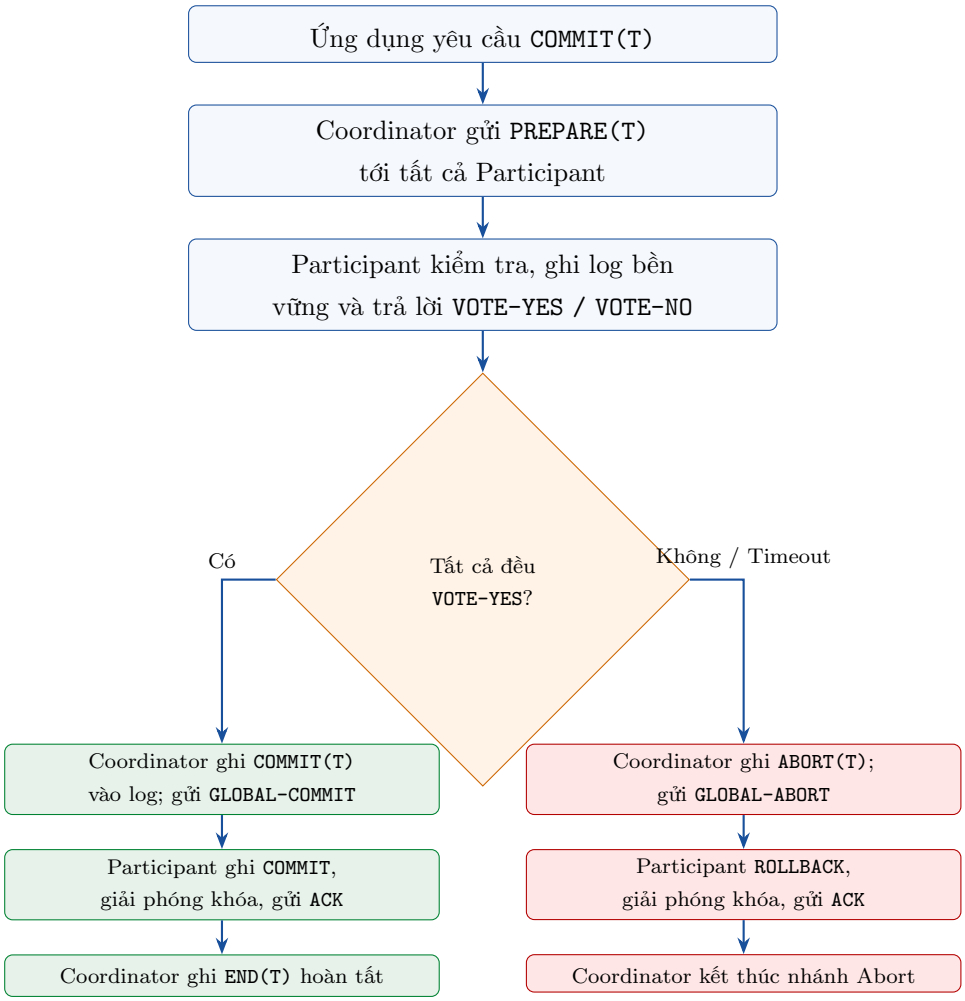
4.3.3 Pha 2 – DECISION (Quyết định Cuối cùng)

1. Nếu tất cả Participant đều `VOTE-YES`, Coordinator ghi quyết định `GLOBAL-COMMIT(T)` bền vững vào log trước khi gửi thông điệp `COMMIT`.
2. Nếu có bất kỳ `VOTE-NO` nào hoặc hết thời gian chờ mà chưa đủ phiếu, Coordinator chọn `GLOBAL-ABORT(T)` và ghi log.
3. Coordinator phát thông điệp `COMMIT/ABORT`. Các Participant thực hiện quyết định, ghi log cục bộ, giải phóng khóa và gửi phản hồi `ACK`.

4. Sau khi nhận đủ `ACK`, Coordinator ghi bản ghi `END(T)` và dọn dẹp trạng thái điều phối.

Trong trường hợp lý tưởng có n Participant, giao thức trao đổi khoảng $4n$ thông điệp giữa Coordinator và các Participant, làm tăng độ trễ đáng kể so với việc commit trên một máy cục bộ.

4.3.4 Lưu đồ thuật toán 2PC



Hình 4.1: Lưu đồ hoạt động của Giao thức Cam kết Hai pha (2PC)

4.3.5 Bảng trạng thái và thông điệp trong 2PC

4.3.6 Ví dụ chi tiết: Giao dịch chuyển tiền giữa hai máy chủ

Xét tài khoản $A = 5.000.000$ đ tại trạm S_1 và $B = 3.000.000$ đ tại trạm S_2. Giao dịch T chuyển 1.000.000 đ từ A sang B. Kết quả mong muốn sau commit: $A = 4.000.000$ đ, $B = 4.000.000$ đ; tổng số dư bảo toàn 8.000.000 đ.

Bảng 4.3: Các trạng thái cốt lõi của nút trong giao thức 2PC

Vai trò nút	**Trạng thái**	**Ý nghĩa / Hành động tương ứng**
Coordinator	`WAITING / COLLECTING`	Đã phát thông điệp PREPARE, đang chờ thu thập phiếu bầu.
Coordinator	`COMMITTED / ABORTED`	Quyết định toàn cục đã ghi log bền vững; gửi lại khi cần.
Participant	`INIT / ACTIVE`	Đang thực hiện các thao tác giao dịch cục bộ.
Participant	`PREPARED`	Đã ghi bền vững phiếu YES; không được tự ý quyết định.
Participant	`COMMITTED / ABORTED`	Đã áp dụng quyết định cuối cùng, giải phóng tài nguyên.

Bảng 4.4: Tiến trình thực thi giao dịch chuyển tiền qua 2PC

Thời điểm	**Coordinator**	S_1 **(Tài khoản** A**)**	S_2 **(Tài khoản** B**)**
t_1	Khởi tạo giao dịch T.	Trừ 1.000.000 đ (chưa commit).	Cộng 1.000.000 đ (chưa commit).
t_2	Gửi thông điệp `PREPARE`.	Ghi log `PREPARED`, trả lời `VOTE-YES`.	Ghi log `PREPARED`, trả lời `VOTE-YES`.
t_3	Nhận đủ YES; ghi log `GLOBAL-COMMIT`.	Chờ quyết định toàn cục.	Chờ quyết định toàn cục.
t_4	Phát thông điệp `COMMIT`.	Commit cục bộ, nhả khóa, gửi `ACK`.	Commit cục bộ, nhả khóa, gửi `ACK`.
t_5	Ghi bản ghi `END(T)`.	Hoàn tất ($A = 4.000.000$ đ).	Hoàn tất ($B = 4.000.000$ đ).

Nếu trạm S_2 gặp sự cố không thể chuẩn bị và bỏ phiếu `VOTE-NO`, Coordinator sẽ phát `GLOBAL-ABORT`. Trạm S_1 hoàn tác khoản trừ, trạm S_2 hoàn tác khoản cộng, dữ liệu khôi phục về trạng thái nguyên vẹn ban đầu.

4.4 Xử Lý Lỗi Và Phục Hồi (Recovery) Trong 2PC

4.4.1 Vai trò của nhật ký bền vững (Write-Ahead Logging)

Write-Ahead Logging (WAL) yêu cầu toàn bộ thông tin trạng thái phải được lưu bền vững xuống đĩa trước khi gửi thông điệp ra mạng:

- `Coordinator: START_2PC(T) -> COMMIT(T) -> END(T)`

- `Participant: UPDATE(T, before, after) -> PREPARED(T) -> COMMIT(T)`
- `Nhánh hủy: UPDATE(T, before, after) -> ABORT(T)`

Mỗi thông điệp gửi lại khi phục hồi phải có tính bất biến lặp (*Idempotent*) theo mã giao dịch để tránh thực thi cập nhật trùng lặp.

4.4.2 Phân tích các kịch bản sự cố và khả năng phục hồi

4.4.3 Tại sao 2PC là giao thức Blocking?

Xét trạm S_1 đã ghi `PREPARED(T)` và gửi `VOTE-YES`. Coordinator mất kết nối trước khi S_1 nhận được kết quả. S_1 không thể tự ý COMMIT (vì trạm khác có thể đã bỏ phiếu NO), cũng không thể tự ý ABORT (vì Coordinator có thể đã quyết định COMMIT và gửi cho trạm khác). Trạm S_1 bị rơi vào trạng thái bế tắc treo (*In-doubt state*), buộc phải giữ chặt các khóa tài nguyên, khiến các giao dịch khác đến sau bị nghẽn dây chuyền.

4.4.4 Lưu đồ phục hồi tại Participant

4.5 Giao Thức Cam Kết Ba Pha (Three-Phase Commit – 3PC)

4.5.1 Động cơ và các giai đoạn

3PC được đề xuất nhằm loại bỏ hiện tượng bị chặn không giới hạn (*Non-blocking*) của 2PC dưới giả định mạng không bị phân hoạch tùy ý và có giới hạn trễ xác định:

1. **Giai đoạn 1 (CanCommit):** Coordinator gửi thông điệp `CAN-COMMIT` thăm dò. Participant kiểm tra tài nguyên và trả lời `VOTE-YES` (chuyển sang trạng thái `READY`) hoặc `VOTE-NO`. Nếu có phiếu NO, giao dịch hủy ngay.
2. **Giai đoạn 2 (PreCommit):** Khi nhận đủ YES, Coordinator gửi `PRE-COMMIT`. Participant nhận lệnh, ghi log bền vững trạng thái `PRECOMMIT` và gửi lại `ACK`. Giai đoạn này đảm bảo tất cả các nút đều đã biết trước rằng giao dịch đã đủ điều kiện cam kết.
3. **Giai đoạn 3 (DoCommit):** Coordinator thu thập đủ ACK, ghi log bền vững `COMMIT` và gửi thông điệp `DO-COMMIT`. Các nút tiến hành commit cục bộ và nhả tài nguyên.

4.5.2 Phục hồi và giới hạn kỹ thuật của 3PC

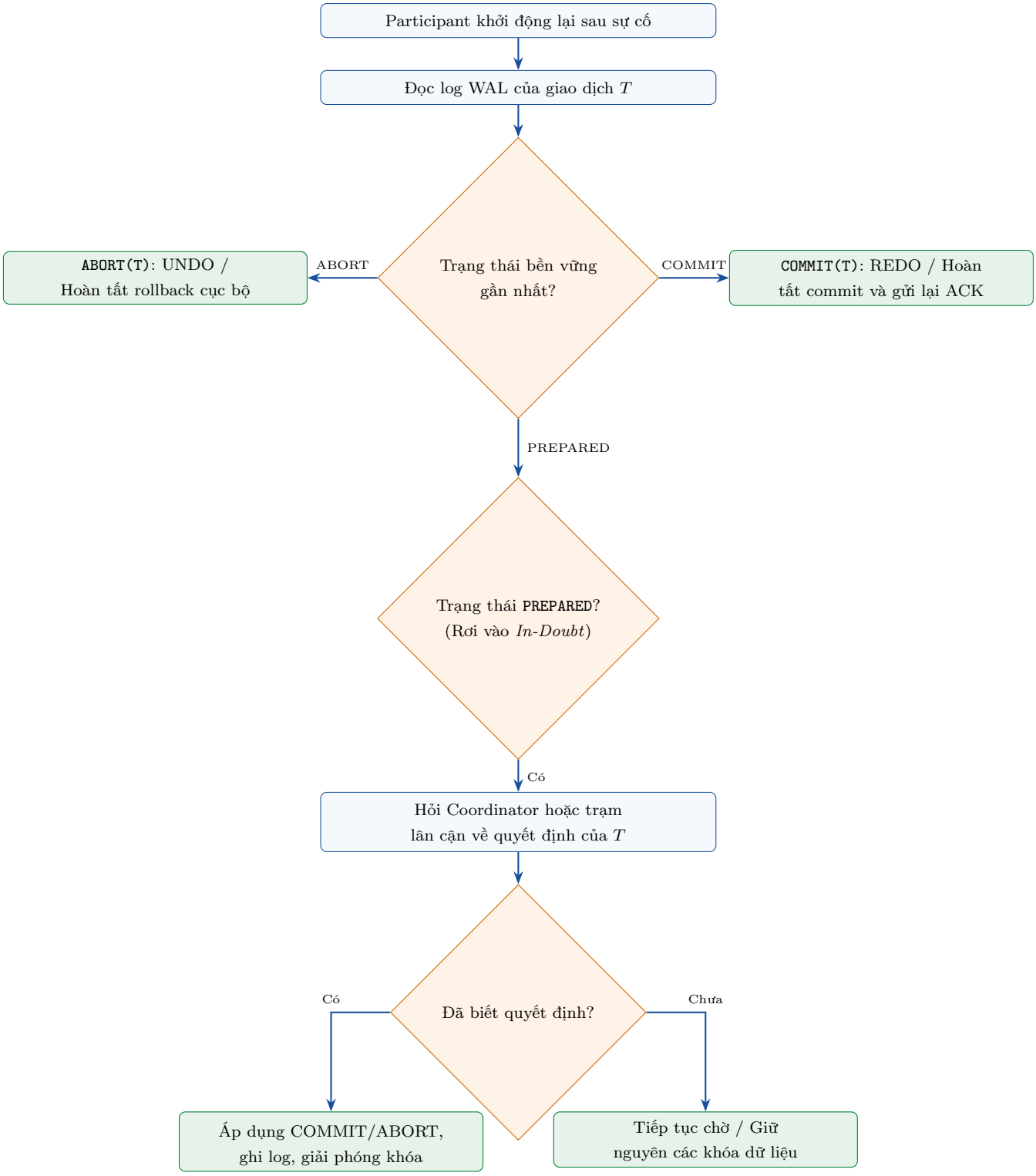
Nhờ có trạng thái trung gian `PRECOMMIT`, nếu Coordinator bị sập:

Bảng 4.5: Phân tích các kịch bản sự cố trong 2PC

Kịch bản sự cố	**Quy trình xử lý hợp lệ**	**Khả năng bị chặn (Blocking)**
Participant sập trước `VOTE-YES`	Coordinator không nhận đủ phiếu sẽ timeout và phát ABORT; Participant khi phục hồi dùng log rollback.	Thường không kéo dài sau khi quyết định hủy được phát.
Participant sập ngay sau `VOTE-YES`	Lúc khởi động lại thấy log PREPARED, bắt buộc phải tra cứu kết quả toàn cục để tuân theo COMMIT/ABORT.	Có thể bị chặn nếu không liên lạc được với nút nắm quyết định.
Participant sập sau khi nhận COMMIT trước ACK	Đọc log COMMIT/REDO nếu cần; gửi lại ACK sau khi phục hồi.	Không làm đổi quyết định; chỉ chậm trễ xác nhận dọn dẹp.
Coordinator sập trước khi ghi quyết định	Sau phục hồi, nếu chưa có COMMIT bền vững, hệ thống quyết định ABORT và thông báo cho các nút.	Participant ở trạng thái PREPARED phải chờ Coordinator bật lại.
Coordinator sập sau khi đã ghi COMMIT bền vững	Sau phục hồi, đọc log thấy COMMIT sẽ gửi lại thông điệp COMMIT cho các nút chưa xác nhận.	Các nút chưa biết quyết định phải chờ trong thời gian mất liên lạc.
Mạng bị phân hoạch (Network Partition)	Nút PREPARED không được tự ý đổi quyết định từ timeout; bắt buộc phải giữ tài nguyên chờ kết nối lại.	**Bị chặn (Blocking)**; thể hiện rõ hạn chế cố hữu của 2PC.
Nhiều nút sập đồng thời mất log	Khôi phục từ các bản sao lưu chuyên biệt; có nguy cơ không thể tự động khôi phục.	Nguy cơ gián đoạn nghiêm trọng.

- *Nếu chưa có nút nào tới PreCommit:* Các nút còn sống có thể bầu Coordinator mới và an toàn quyết định `ABORT`.
- *Nếu đã có ít nhất một nút nhận PreCommit:* Các nút còn sống biết rằng không có nút nào bỏ phiếu NO, do đó có thể điều phối tiếp tục tiến trình để hoàn tất `COMMIT`.

Giới hạn thực tế: 3PC đòi hỏi giả định đồng bộ rất mạnh về mạng. Khi mạng xảy ra phân hoạch (*Network Partition*), 3PC vẫn có thể đưa ra quyết định sai lệch ở hai nửa mạng,



Hình 4.2: Lưu đồ phục hồi giao dịch 2PC tại Participant

do đó trong thực tế các hệ thống dữ liệu hiện đại ưa chuộng các thuật toán đồng thuận như Paxos hoặc Raft hơn.

4.5.3 Bảng so sánh đối chiếu giữa 2PC và 3PC

Bảng 4.6: So sánh đặc tính kỹ thuật giữa giao thức 2PC và 3PC

Tiêu chí	**Giao thức 2PC**	**Giao thức 3PC**
Số giai đoạn chính	2 pha: Prepare, Decision.	3 pha: CanCommit, PreCommit, DoCommit.
Độ phức tạp thông điệp	Ít thông điệp hơn ($4n$), độ trễ thấp hơn.	Tăng thêm thông điệp ($6n$) và nhiều lần ghi đĩa log hơn.
Coordinator sập khi đang chờ	Participant rơi vào trạng thái In-doubt và bị chặn (*Blocking*).	Có thể giải phóng trạng thái nghẽn nhờ pha trung gian PreCommit.
Phân hoạch mạng (Partition)	Bảo toàn an toàn quyết định nhưng đánh đổi tính sẵn sàng.	Không bảo đảm an toàn khi mạng bị chia cắt tùy ý.
Ứng dụng thực tế	Được ứng dụng rộng rãi trong các chuẩn XA, SQL truyền thống.	Chủ yếu mang giá trị nghiên cứu học thuật.

4.6 Kiểm Soát Đồng Thời Phân Tán

4.6.1 Vì sao cần kiểm soát đồng thời?

Khi nhiều giao dịch thực thi xen kẽ trên nhiều trạm, các hiện tượng mất cập nhật (*Lost Update*) hoặc đọc dữ liệu rác (*Dirty Read*) sẽ phá vỡ tính đúng đắn của CSDL. Mục tiêu lý thuyết quan trọng là bảo đảm tính **Khả tuần tự toàn cục (Global Serializability)**: kết quả thực thi đồng thời phải tương đương với một trật tự tuần tự nào đó.

4.6.2 Khóa hai pha phân tán (Distributed Two-Phase Locking – 2PL)

Quy tắc 2PL gồm hai pha: pha tăng trưởng (chỉ xin thêm khóa, không nhả khóa) và pha thu hẹp (chỉ nhả khóa, không xin thêm).

- Trong hệ phân tán, giao dịch phải tuân thủ nghiêm ngặt quy tắc này trên toàn bộ các trạm mà nó thao tác.
- **Strict 2PL:** Yêu cầu giữ toàn bộ khóa độc quyền (*Exclusive Locks* – X) cho đến khi giao dịch hoàn tất `COMMIT/ABORT`, ngăn chặn đọc dữ liệu chưa cam kết.

4.6.3 Điều khiển theo nhãn thời gian (Timestamp Ordering – TO)

Mỗi giao dịch T được cấp một nhãn thời gian duy nhất TS(T). Đối với mỗi mục dữ liệu X, hệ thống ghi nhận nhãn đọc lớn nhất read_TS(X) và nhãn ghi lớn nhất write_TS(X):

Bảng 4.7: Quy tắc kiểm tra xung đột trong giải thuật Timestamp Ordering

Thao tác yêu cầu	**Điều kiện xảy ra xung đột**	**Biện pháp xử lý của hệ thống**
T **yêu cầu** `READ(X)`	TS(T) < write_TS(X)	T đến muộn sau một thao tác ghi mới hơn; giao dịch T bị hủy và cấp nhãn lại.
T **yêu cầu** `WRITE(X)`	TS(T) < read_TS(X) hoặc TS(T) < write_TS(X)	Thao tác ghi vi phạm thứ tự thời gian; giao dịch T bị hủy và cấp nhãn lại.

4.6.4 Phát hiện và xử lý bế tắc toàn cục (Global Deadlock Detection)

Bế tắc xảy ra khi tồn tại một chu trình chờ tài nguyên giữa các giao dịch. Đồ thị chờ (*Wait-For Graph – WFG*) phân tán có đỉnh là giao dịch, cạnh có hướng $T_i \to T_j$ thể hiện T_i đang chờ tài nguyên bị giữ bởi T_j.

- *Ví dụ:* Tại trạm S_1, T_1 giữ $X(A)$ và T_2 chờ $A \implies T_2 \to T_1$. Tại trạm S_2, T_2 giữ $X(B)$ và T_1 chờ $B \implies T_1 \to T_2$.
- Mỗi trạm cục bộ chỉ thấy một nhánh không tạo thành chu trình. Chỉ khi điều phối viên thu thập và hợp nhất thành **Đồ thị WFG Toàn cục**, chu trình $T_1 \to T_2 \to T_1$ mới lộ diện. Hệ thống sẽ chọn một giao dịch làm nạn nhân (*Victim*) để rollback và giải phóng khóa.

4.7 Định Lý CAP Và Sự Đánh Đổi Trong Thực Tế

4.7.1 Phát biểu và cách hiểu chính xác

Theo công trình hình thức hóa của Gilbert và Lynch [**?**], định lý CAP xét 3 thuộc tính:

- **C (Consistency):** Nhất quán mạnh (tương đương *Linearizability*), mọi thao tác đọc đều nhận được dữ liệu ghi mới nhất.

- **A (Availability):** Mọi nút mạng không bị lỗi đều phải trả về phản hồi hợp lệ cho các yêu cầu.
- **P (Partition Tolerance):** Hệ thống vẫn hoạt động ngay cả khi các gói tin truyền thông giữa các cụm nút bị thất lạc hoặc trễ vô hạn.

Bản chất kỹ thuật: Không nên hiểu máy móc là "được chọn 2 trong 3 ở mọi thời điểm". Trong thực tế hạ tầng mạng, sự cố phân hoạch (P) là bất khả kháng; do đó khi có phân hoạch xảy ra, hệ thống bắt buộc phải đánh đổi giữa tính nhất quán (C) và tính sẵn sàng (A).

4.7.2 Ví dụ phân hoạch với hai bản sao dữ liệu

Giả sử bản sao R_1 và R_2 cùng có $x = 0$. Mạng bị đứt kết nối giữa hai trạm:

- Client tại R_1 ghi $x = 1$ thành công.
- Sau đó, Client tại R_2 yêu cầu đọc x:
  - Nếu R_2 trả về $x = 0 \implies$ Đạt Availability nhưng vi phạm Consistency (trả dữ liệu cũ).
  - Nếu R_2 từ chối hoặc chờ kết nối lại $\implies$ Bảo toàn Consistency nhưng mất Availability.

4.7.3 Bảng so sánh xu hướng thiết kế CP và AP

Bảng 4.8: So sánh giữa hai khuynh hướng thiết kế hệ thống CP và AP

Tiêu chí	**Hệ thống thiên về CP**	**Hệ thống thiên về AP**
Ưu tiên khi phân hoạch	Bảo vệ nhất quán mạnh cho dữ liệu.	Duy trì phản hồi phục vụ liên tục.
Ứng xử ở vùng mất kết nối	Chờ đợi hoặc từ chối thao tác.	Chấp nhận đọc/ghi trên bản sao cục bộ.
Rủi ro chính	Tăng độ trễ, suy giảm tính sẵn sàng.	Bản sao phân kỳ, phải giải quyết xung đột sau.
Nghiệp vụ phù hợp	Tài chính, chuyển tiền, quản lý tồn kho.	Mạng xã hội, bảng tin, giỏ hàng thương mại.
Hệ quản trị tiêu biểu	Google Spanner, CockroachDB, HBase.	Apache Cassandra, Amazon DynamoDB, CouchDB.

4.7.4 Liên hệ giữa CAP, 2PC và mô hình BASE

Giao thức 2PC khi gặp phân hoạch mạng sẽ chọn dừng giao dịch (blocking) để giữ an toàn nguyên tử, do đó mang thiên hướng CP. Ngược lại, để đạt độ mở rộng quy mô cực lớn trên Internet, nhiều hệ thống chuyển dịch từ mô hình ACID sang mô hình **BASE** (*Basically Available, Soft-state, Eventual consistency*), chấp nhận trạng thái nhất quán sau cùng (*Eventual Consistency*) để tối đa hóa tính sẵn sàng.

4.8 Đánh Giá Tổng Hợp Và Đề Xuất Thiết Kế

Bảng 4.9: Phân loại các kỹ thuật quản lý giao dịch theo lớp chức năng

Bài toán kỹ thuật	**Giải pháp tiêu biểu**	**Đánh đổi nổi bật**
Thống nhất cam kết đa trạm	2PC	Đảm bảo Atomic Commit nhưng chịu rủi ro Blocking và trễ $4n$.
Giảm rủi ro treo hệ thống	3PC	Thêm pha PreCommit ($6n$) nhưng đòi hỏi giả định mạng đồng bộ.
Ngăn ngừa xen kẽ sai	Distributed Strict 2PL	Đạt chuẩn khả tuần tự nhưng phát sinh hàng đợi chờ khóa.
Sắp lịch theo trật tự logic	Timestamp Ordering	Không gây bế tắc khóa nhưng có tỷ lệ Rollback khi xung đột cao.
Nhận diện chu trình chờ	Global WFG	Phát hiện bế tắc chính xác nhưng chịu chi phí ghép đồ thị mạng.
Đánh đổi khi phân hoạch	Phân tích CAP	Lựa chọn giữa nhất quán dữ liệu hay tính sẵn sàng nghiệp vụ.

Phụ Lục: Mã Giả Và Bộ Tình Huống Kiểm Thử

Phụ lục A: Mã giả Thuật toán Cam kết Hai pha (2PC)

```
COORDINATOR_COMMIT(T, participants):
    send PREPARE(T) to every participant
    wait for votes or timeout
    if every participant replied YES:
        force_log(COMMIT, T)  -- Ghi log ben vung truoc khi gui
        decision = COMMIT
    else:
        force_log(ABORT, T)
        decision = ABORT
    repeat:
        send decision(T) to participants not yet acknowledged
        wait for ACKs or retry trigger
    until required ACKs received
    log END(T)
```

```
ON_PREPARE(T):
    if local_checks_pass and recovery_state_can_be_persisted:
        force_log(PREPARED, T)
        send VOTE_YES(T)
        state[T] = PREPARED
    else:
        rollback_local(T)
        log ABORT(T)
        send VOTE_NO(T)

ON_DECISION(T, decision):
    if decision == COMMIT:
        commit_local_once(T)
```

```
        log COMMIT(T)
    else:
        rollback_local_once(T)
        log ABORT(T)
    release_held_locks(T)
    send ACK(T)
```

```
ON_RESTART(T):
    state = read_durable_log(T)
    if state == COMMIT:
        redo_if_needed(T)
        send ACK(T)
    else if state == ABORT:
        undo_if_needed(T)
    else if state == PREPARED:
        repeat
            query_known_decision(T)
        until verified result found
        apply_result_once(T)  -- TUYET DOI KHONG tu y abort
    else:
        rollback_uncommitted_local_changes(T)
```

Phụ lục B: Bộ Tình huống Kiểm thử Lý thuyết 2PC & CAP

Bảng 4.10: Bộ tình huống kiểm thử lý thuyết cho giao dịch và phân hoạch mạng

Mã TC	**Tình huống kiểm thử**	**Hành vi và Kết quả mong đợi**
TC01	Tất cả các trạm đều `VOTE-YES`	Quyết định `GLOBAL-COMMIT` ghi bền vững; toàn bộ trạm commit thành công.
TC02	Một trạm trả lời `VOTE-NO`	Coordinator phát `GLOBAL-ABORT`; mọi thay đổi cục bộ được hoàn tác sạch sẽ.
TC03	Participant sập sau khi `PREPARED`	Khi bật lại, đọc log thấy PREPARED sẽ liên hệ hỏi quyết định; không tự ý abort.
TC04	Coordinator sập sau khi ghi COMMIT	Khi phục hồi, đọc log và phát lại COMMIT; Participant xử lý theo tính chất lặp an toàn.
TC05	Phân hoạch mạng khi các trạm đã PREPARED	Các trạm chưa rõ quyết định tiếp tục chờ và giữ khóa; bảo vệ tính nguyên tử của giao dịch.
TC06	Hai giao dịch T_1, T_2 chờ khóa chéo	Đồ thị WFG toàn cục nhận diện chu trình bế tắc; chọn một giao dịch làm nạn nhân để rollback.
TC07	Phân hoạch mạng trên hai bản sao	Hệ thống CP từ chối phục vụ để bảo vệ nhất quán; hệ thống AP chấp nhận dữ liệu có thể cũ.

Chương 5

Tổng Kết Và Hướng Phát Triển

5.1 Tổng Kết Kết Quả Nghiên Cứu

Sau quá trình nghiên cứu chuyên sâu về đề tài B6 dựa trên các giáo trình quốc tế kinh điển (*Özsu & Valduriez, Ramakrishnan & Gehrke*), nhóm đã hoàn thành toàn diện các mục tiêu lý thuyết cốt lõi trên cả ba phương diện:

- **Kiến trúc hệ thống và Thiết kế phân mảnh (Chương 1 & 2):** Làm chủ mô hình phân tầng ANSI/SPARC mở rộng (5 tầng) và 4 mức độ trong suốt phân tán. Chuẩn hóa cơ sở toán học của kỹ thuật phân mảnh ngang nguyên thủy thông qua tập vị từ cơ sở Pr và vị từ hội sơ cấp minterm ($m_i = \bigwedge p_j^*$), phân mảnh ngang dẫn xuất bằng phép nửa nối ($R \ltimes S_i$), phân mảnh dọc với ma trận tương quan AAM và thuật toán BEA, đồng thời bảo đảm nghiêm ngặt 3 quy tắc bắt buộc: *Đầy đủ, Tái tạo* và *Tách biệt*.

- **Quy trình xử lý và Tối ưu hóa truy vấn phân tán (Chương 3):** Nắm vững tiến trình 3 giai đoạn: *Phân rã truy vấn* (chuẩn hóa CNF/DNF, đẩy sớm phép chọn/chiếu) $\rightarrow$ *Bản địa hóa dữ liệu* (cắt tỉa phân mảnh rỗng qua kiểm tra mâu thuẫn vị từ) $\rightarrow$ *Tối ưu hóa toàn cục* (lựa chọn trạm thực thi và thuật toán nối). Xây dựng bài toán định lượng minh chứng kỹ thuật Semi-join có khả năng cắt giảm tới $99,5\%$ dung lượng truyền mạng khi độ chọn lọc của phép nối thấp.

- **Giao dịch và Kiểm soát nhất quán phân tán (Chương 4):** Làm rõ cơ chế cam kết nguyên tử thông qua giao thức 2PC/3PC kết hợp nhật ký ghi trước (WAL) và phân tích nguyên nhân gây bế tắc treo (Blocking) của 2PC. Hệ thống hóa các giải thuật kiểm soát tương tranh (Distributed Strict 2PL, Timestamp Ordering, nhận diện chu trình WFG toàn cục) và xác lập ranh giới đánh đổi giữa tính nhất quán mạnh và tính sẵn sàng theo Định lý CAP.

5.2 Những Nhận Định Và Bài Học Cốt Lõi Rút Ra

Từ các kết quả phân tích lý thuyết, nhóm rút ra 3 nguyên lý thiết kế then chốt cho hệ cơ sở dữ liệu phân tán:

1. **Mạng truyền thông là nút thắt cổ chai chi phối:** Khác với CSDL tập trung tối ưu hóa theo chi phí I/O đĩa, mọi quyết định thiết kế trong DDBMS (từ phân mảnh dữ liệu, cấp phát trạm, đến thuật toán Semi-join) đều xoay quanh mục tiêu tối thiểu hóa lượng dữ liệu trung gian và số lượt truyền thông điệp qua mạng.
2. **Sự đánh đổi không thể tránh khỏi giữa Đọc và Ghi:** Mô hình nhân bản toàn phần triệt tiêu độ trễ của luồng đọc ($C_{\text{mạng}} = 0$) và tối ưu hóa tính sẵn sàng, nhưng lại khiến chi phí cập nhật ghi bùng nổ do bắt buộc phải điều phối cam kết đồng bộ trên toàn mạng.
3. **Không tồn tại kiến trúc hoàn hảo tuyệt đối:** Định lý CAP khẳng định khi xảy ra phân hoạch mạng, hệ thống bắt buộc phải lựa chọn giữa tính nhất quán dữ liệu (CP - ưu tiên nguyên tử, chấp nhận dừng giao dịch) hoặc tính sẵn sàng dịch vụ (AP - ưu tiên phản hồi liên tục, chấp nhận mô hình nhất quán sau cùng BASE).

5.3 Định Hướng Triển Khai Thực Nghiệm

Nền tảng lý thuyết phân tán vững chắc ở đề tài B6 là tiền đề kỹ thuật trực tiếp để nhóm triển khai giai đoạn thực nghiệm tiếp theo ở **Đề tài A2: Thiết lập nhân bản 1 Master, 2 Slave; áp dụng thử nghiệm với CSDL quản lý bán hàng tại 1 siêu thị**. Các công việc cụ thể bao gồm:

1. **Xây dựng môi trường hạ tầng phân tán:** Thiết lập cụm 3 nút mạng độc lập (1 máy chủ Master và 2 máy chủ Slaves) trên nền tảng ảo hóa (Docker containers) kết nối mạng nội bộ.
2. **Cấu hình cơ chế sao chép dữ liệu (Replication):** Triển khai hệ quản trị MySQL, cấu hình đồng bộ dữ liệu dựa trên nhật ký nhị phân từ Master sang Relay Log của các Slaves.
3. **Thiết kế CSDL và Thử nghiệm phân tải:** Cài đặt lược đồ CSDL siêu thị; hiện thực cơ chế phân luồng nghiệp vụ: định tuyến toàn bộ thao tác ghi về Master và chia tải đọc cho 2 Slaves.
4. **Đo đạc và Đánh giá sự cố:** Thực nghiệm đo độ trễ sao chép (*Replication Lag*) dưới tải cao, kiểm thử kịch bản chịu lỗi khi rớt mạng hoặc khi một nút Slave bị sập để đánh giá tính sẵn sàng thực tế của hệ thống.

Tài liệu tham khảo

[1] M. Tamer Özsu, Patrick Valduriez (2020), *Principles of Distributed Database Systems*, 4th Edition, Springer.

[2] Raghu Ramakrishnan, Johannes Gehrke (2003), *Database Management Systems*, 3rd Edition, McGraw-Hill.

[3] Abraham Silberschatz, Henry F. Korth, S. Sudarshan (2019), *Database System Concepts*, 7th Edition, McGraw-Hill.

[4] Ramez Elmasri, Shamkant B. Navathe (2015), *Fundamentals of Database Systems*, 7th Edition, Pearson.